МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ

ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ

ВЫСШЕГО ОБРАЗОВАНИЯ

«НОВОСИБИРСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»

Кафедра защиты информации



РАСЧЕТНО-ГРАФИЧЕСКАЯ РАБОТА

по дисциплине: «Безопасность систем баз данных»

по теме: «Защита БД студий дубляжа»

Выполнил:

Студент гр. «АБс-322», «АВТФ»

Бутаков Никита Максимович

«___» _____ 2025г

_____

(подпись)

Проверил:

Ассистент кафедры ЗИ

Питько Яна Андреевна

«___» _____ 2025г

_____

(подпись)

Новосибирск 2025г.

Содержание

Задание к РГЗ 3
Описание предметной области и ролей участников 4
ER-модель и классификация 5
DDL и схемы 10
Роли и привилегии 11
Логирование подключений пользователя 12
SECURITY DEFINER-функции 13
Сравнение CHECK vs триггер 15
Включение RLS и политики 16
Передача контекста 17
Безопасные представления 19
История изменений в audit.row_change_log 21
Анализ производительности 22
Автоматическое резервное копирование audit-логов 23
Система временных привилегий 25
Вывод 28
Приложение А 29
Приложение Б 73

Задание к РГЗ

Реализуйте систему временных привилегий (Just-In-Time Access). Создайте механизм, позволяющий роли «app_writer» временно (на N минут) получать право на выполнение чувствительной операции (например, удаление записи) через вызов SECURITY DEFINER-функции (request_temp_privilege (operation_name TEXT, duration_min INT).

Функция должна: проверять, разрешена ли операция для роли; записывать запрос в «audit.temp_access_log» (request_time, caller_role, operation, expires_at); устанавливать временную метку истечения привилегии (через GUC «app.temp_privilege_until» или временную таблицу); при каждой чувствительной операции – проверять, активна ли привилегия; после истечения – автоматически блокировать операцию.

Протестируйте: запрос → выполнение → попытка после истечения → ошибка.

Описание предметной области и ролей участников

Предметная область база данных студии дубляжа, предназначенная для управления авторскими материалами, контрактами с актёрами, расписаниями записей и защищёнными архивами аудиофайлов.

Ролевая модель разработана по принципу разделения обязанностей и наименьших привилегий. Доступ к данным сегментирован по филиалам/студиям. В таблице 1 представлено отношение роли и привилегии. Код реализации представлен в приложении А.

Таблица 1 – Ролевая модель

Роль	**Привилегии**
app_reader	USAGE для схем app, ref; SELECT во всех таблицах схем app, ref
app_writer	USAGE для схем app, ref; SELECT, INSERT, UPDATE во всех таблицах схем app, ref
app_owner	USAGE для схем app, ref, stg; ALL PRIVILEGES на все объекты схем app, ref, stg
auditor	USAGE для схемы audit; SELECT во всех таблицах схемы audit
ddl_admin	USAGE для схем app, ref, audit, stg; CREATE для схем app, ref, audit, stg
dml_admin	USAGE для схем app, ref, stg; SELECT, INSERT, UPDATE, DELETE во всех таблицах схем app, ref, stg
voice_actor	USAGE для схем app, ref; SELECT в таблицах app и ref (только свои данные)

ER-модель и классификация

ER-диаграмма отражает основные сущности предметной области: actors, contracts, projects, recording_sessions, audio_archives и справочники (Рисунок 1).

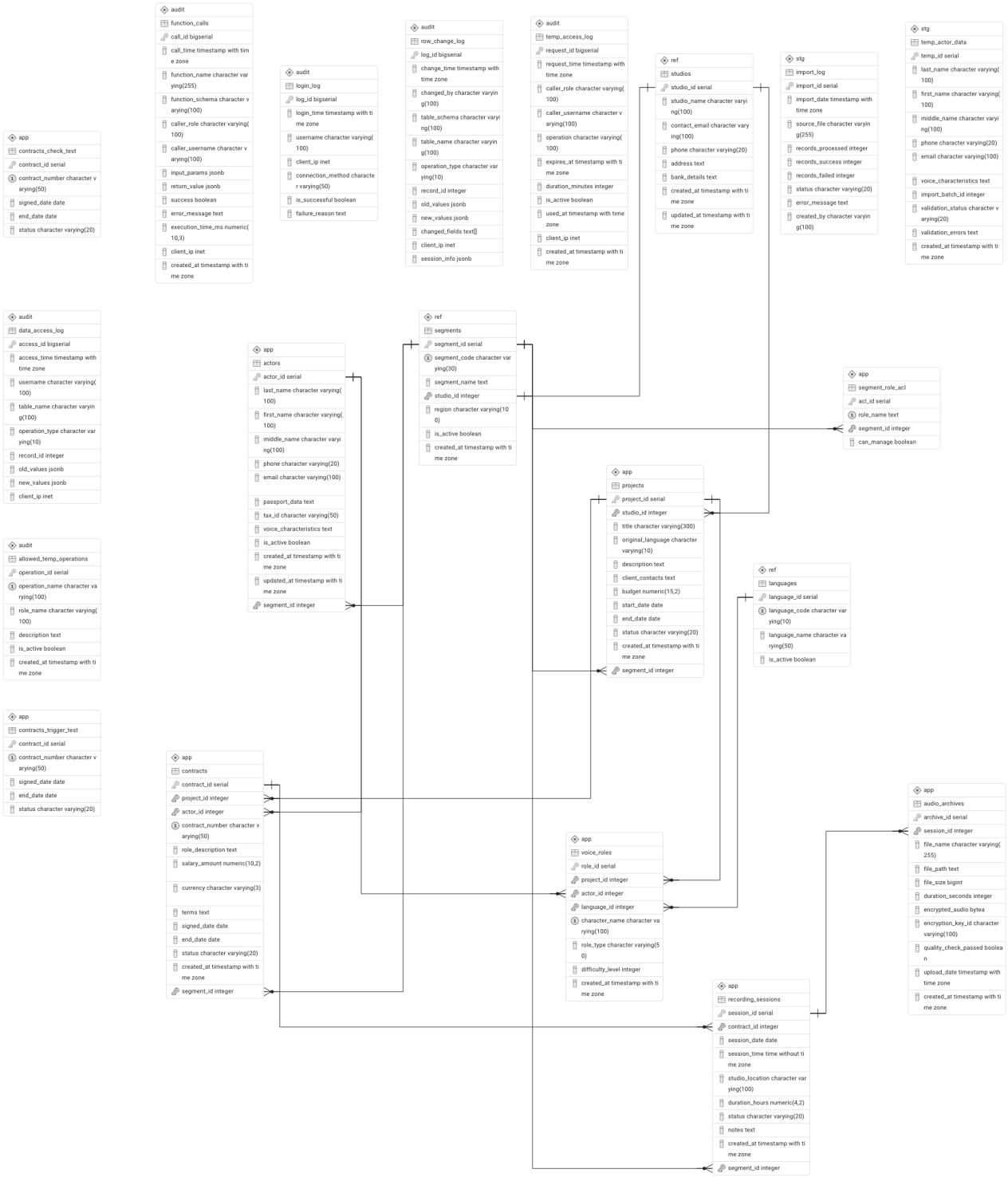


Рисунок 1 – ER-диаграмма

Данные классифицированы по четырем уровням конфиденциальности:

- Public – публичные данные (название проекта, код языка);
- Internal – внутренняя информация (идентификаторы, статусы);
- Confidential – конфиденциальные данные (сумма контракта, бюджет проекта);
- Restricted – данные с ограниченным доступом (паспортные данные, номер телефона).

В таблицах 2-10 представлена классификация каждой таблицы в нашей базе данных.

Таблица 2 – Классификация данных таблицы studios

Колонка	**Класс**
studio_id	Internal
studio_name	Public
contact_email	Internal
phone	Internal
address	Internal

Таблица 3 – Классификация данных таблицы languages

Колонка	**Класс**
language_id	Internal
language_code	Public
language_name	Public
is_active	Internal

Таблица 4 – Классификация данных таблицы actors

Колонка	**Класс**
actor_id	Internal
full_name	Public
phone	Restricted
email	Restricted

Таблица 5 – Классификация данных таблицы projects

Колонка	**Класс**
project_id	Internal
studio_id	Internal
title	Public
original_language	Public

Таблица 6 – Классификация данных таблицы contracts

Колонка	**Класс**
contract_id	Internal
project_id	Internal
actor_id	Internal
contract_number	Internal
role_description	Internal
salary_amount	Confidential
currency	Internal
terms	Confidential
signed_date	Internal

Таблица 7 – Классификация данных таблицы recording_sessions

Колонка	**Класс**
session_id	Internal
contract_id	Internal
session_date	Internal
session_time	Internal
studio_location	Internal
duration_hours	Internal
status	Internal
notes	Internal

Таблица 8 – Классификация данных таблицы audio_archives

Колонка	**Класс**
archive_id	Internal
session_id	Internal
file_name	Internal
file_path	Internal

Таблица 9 – Классификация данных таблицы voice_roles

Колонка	**Класс**
role_id	Internal
project_id	Internal
actor_id	Internal

Таблица 10 – Классификация данных таблицы login_log

Колонка	**Класс**
log_id	Internal
login_time	Confidential
username	Confidential
client_ip	Restricted
connection_method	Internal
is_successful	Internal
failure_reason	Internal

DDL и схемы

База данных состоит из четырех логических схем:

- app – основные бизнес-данные (actors, contracts, projects и др.);
- ref – справочники (studios, languages, segments);
- audit – таблицы для логирования (login_log, row_change_log);
- stg – промежуточные таблицы для загрузки данных.

В таблице 11 отображено соотношение схем к таблицам.

Таблица 11 – Определение схем и таблиц

Схема	**Таблица**
app	actors
app	projects
app	contracts
app	recording_sessions
app	audio_archives
app	voice_roles
ref	studios
ref	languages
audit	login_log
stg	import_log

Все таблицы содержат первичные ключи (PK), внешние ключи (FK), ограничения NOT NULL, UNIQUE и CHECK. Для оптимизации производительности созданы индексы по полям, используемым в JOIN и фильтрации.

Также были сгенерированы тестовые данные (≥ 10 строк) в таких ключевых таблицах, как actors (актёры озвучивания), contracts (контракты с актёрами), recoring_sessions (сессии записи), audio_archives (аудиоархивы), voice_roles (роли озвучивания). Код реализации представлен в приложении А.

Роли и привилегии

Реализована система ролевого управления доступом:

Все привилегии PUBLIC отозваны как на уровне базы данных, так и на уровне таблиц и схем. Для каждой роли явно назначены привилегии на уровне схем и таблиц. Настроены DEFAULT PRIVILEGES для автоматического назначения прав на новые объекты. Доступ к схеме audit строго ограничен: прямая модификация (INSERT, UPDATE, DELETE) запрещена. Все записи в таблицы аудита выполняются исключительно через триггеры и SECURITY DEFINER-функции. Код реализации представлен в приложении А.

Логирование подключений пользователя

В нашей базе данных реализована система аудита. Функция audit.log_connection() автоматически записывает все подключения пользователей с фиксацией имени, IP-адреса и статуса авторизации (Рисунок 2).

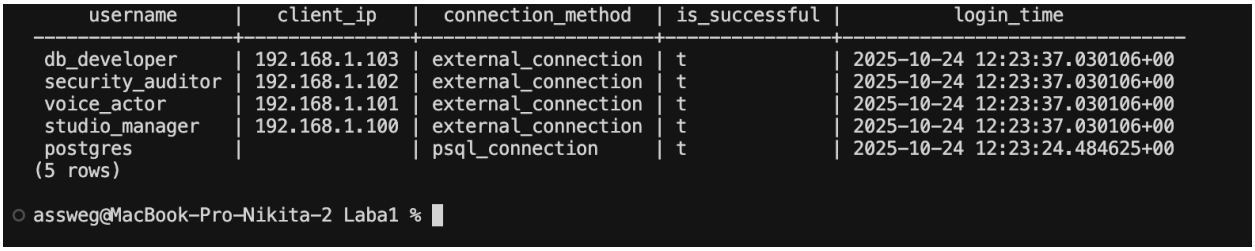


Рисунок 2 – Вывод логов аутентификации пользователей

На рисунке 2 вывод таблицы аудита, в которой показываются данные логирования захода в систему.

SECURITY DEFINER-функции

Реализованы SECURITY DEFINER-функции для выполнения чувствительных операций с повышенными привилегиями, но под контролем бизнес-логики.

В качестве чувствительных операций были выбраны:

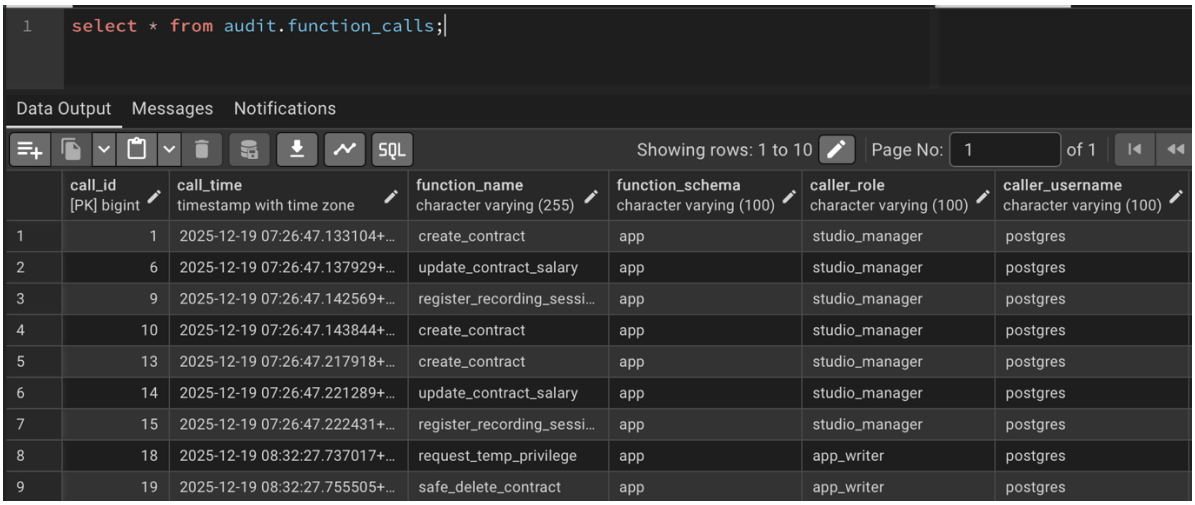
1. Создание контракта с актёром (app.create_contract) – операция создания ключевого бизнес-объекта, содержащего конфиденциальные данные (сумма контракта, условия). Требует повышенных привилегий для проверки целостности данных и логирования.

2. Обновление зарплаты в контракте (app.update_contract_salary) – операция изменения конфиденциальных финансовых данных. Требует строгой валидации и контроля доступа для предотвращения несанкционированных изменений.

3. Регистрация сессии записи (app.register_recording_session) – бизнес-транзакция, регистрирующая сессию записи дубляжа. Требует проверки статуса контракта и валидации параметров сессии для обеспечения целостности бизнес-процессов.

Все вызовы таких функций логируются в таблице audit.function_calls (время вызова, имя функции, вызвавшая роль, параметры, успех, время выполнения).

Также в каждой из функций реализована валидация данных для корректной вставки. Результат вызова одной из функции представлен на рисунке 3.



```
1  select * from audit.function_calls;
```

Data Output | Messages | Notifications

Showing rows: 1 to 10 | Page No: 1 of 1

	call_id [PK] bigint	call_time timestamp with time zone	function_name character varying (255)	function_schema character varying (100)	caller_role character varying (100)	caller_username character varying (100)
1	1	2025-12-19 07:26:47.133104+...	create_contract	app	studio_manager	postgres
2	6	2025-12-19 07:26:47.137929+...	update_contract_salary	app	studio_manager	postgres
3	9	2025-12-19 07:26:47.142569+...	register_recording_sessi...	app	studio_manager	postgres
4	10	2025-12-19 07:26:47.143844+...	create_contract	app	studio_manager	postgres
5	13	2025-12-19 07:26:47.217918+...	create_contract	app	studio_manager	postgres
6	14	2025-12-19 07:26:47.221289+...	update_contract_salary	app	studio_manager	postgres
7	15	2025-12-19 07:26:47.222431+...	register_recording_sessi...	app	studio_manager	postgres
8	18	2025-12-19 08:32:27.737017+...	request_temp_privilege	app	app_writer	postgres
9	19	2025-12-19 08:32:27.755505+...	safe_delete_contract	app	app_writer	postgres

Рисунок 3 – Таблица audit.function_calls

Код реализации SECURITY DEFINER-функции представлен в приложении А.

Сравнение CHECK vs триггер

Для сравнения реализации через CHECK-ограничения и триггеры было выбрано следующее бизнес-правило для таблицы контрактов:

1. Правило валидации дат контракта: дата окончания контракта (end_date) должна быть больше или равна дате подписания (signed_date). Это обеспечивает логическую целостность данных, исключая ситуацию, когда контракт заканчивается раньше, чем был подписан.

2. Правило для завершённых контрактов: если контракт имеет статус completed, то дата окончания (end_date) должна быть установлена и не может быть в будущем (т.е. end_date <= CURRENT_DATE). Это гарантирует, что завершённый контракт не может иметь дату окончания в будущем.

Результаты тестирования на 10,000 строк представлен на рисунке 4.

```
==================================================
ТЕСТ ПРОИЗВОДИТЕЛЬНОСТИ: CHECK vs TRIGGER
==================================================

Запуск теста на 10,000 строк...
Это может занять некоторое время...
 method  | inserted_count | failed_count | total_time_ms | avg_time_per_insert_ms
---------+----------------+--------------+---------------+------------------------
 CHECK   |           9864 |          136 |    133.895000 | 0.01357410786699107867
 TRIGGER |           9899 |          101 |     97.713000 | 0.00987099707041115264
(2 rows)
```

Рисунок 4 – Сравнение CHECK и TRIGGER

Исходя из результатов тестирования производительности, при выполнении операции вставки 10,000 строк с одинаковыми данными были получены следующие показатели:

1. Для метода CHECK-ограничений было успешно вставлено 9,864 записи, при этом 136 операций вставки завершились неудачей. Общее время выполнения составило 133.895 миллисекунд.

2. Для метода триггеров было успешно вставлено 9,899 записей, при этом 101 операции вставки завершились неудачей. Общее время выполнения составило 97.713 миллисекунд. Код реализации представлен в приложении А.

Включение RLS и политики

Реализованная система безопасности данных обеспечивает строгую изоляцию данных между филиалами на уровне отдельных строк таблиц базы данных (RLS). Основой разграничения служит идентификатор филиала.

На все ключевые таблицы, такие как пользователи, контракты и проекты, применены политики безопасности, которые автоматически добавляют условия фильтрации по идентификатору филиала ко всем операциям выборки, вставки, обновления и удаления, гарантируя, что пользователь взаимодействует только с данными своего сегмента. При этом для роли аудитора сделано исключение – ей предоставлен полный доступ ко всем записям без ограничений по филиалам, что позволяет проводить сквозные проверки.

Для операций выборки (SELECT):

- Пользователь видит только записи своего сегмента;
- Используется условие USING с проверкой видимости сегмента.

Для операций модификации (INSERT/UPDATE/DELETE):

- USING - проверка отбора существующих записей;
- WITH CHECK - проверка новых значений при вставке/обновлении;
- Двойная проверка предотвращает кросс-сегментные операции.

Передача контекста

Была разработана безопасная функция set_session_ctx, которая выполняет роль шлюза для установки рабочего контекста пользователя, она устанавливает контекст сегмента и актёра для текущей транзакции. Функция реализована с правами SECURITY DEFINER, что обеспечивает выполнение с повышенными привилегиями при сохранении контроля доступа (Рисунок 5).

```
-- SECURITY DEFINER-функция передачи контекста
CREATE OR REPLACE FUNCTION app.set_session_ctx(p_segment_id INTEGER, p_actor_id INTEGER DEFAULT NULL)
RETURNS VOID
LANGUAGE plpgsql
SECURITY DEFINER
SET search_path = app, ref, public
AS $$
DECLARE
    v_effective_role TEXT;
BEGIN
    v_effective_role := current_setting('role', true);
    IF v_effective_role IS NULL THEN
        v_effective_role := session_user;
    END IF;

    IF NOT EXISTS (
        SELECT 1
        FROM app.segment_role_acl
        WHERE role_name = v_effective_role
          AND segment_id = p_segment_id
    ) THEN
        RAISE EXCEPTION 'Роль % не имеет доступа к сегменту %', v_effective_role, p_segment_id;
    END IF;

    PERFORM set_config('app.segment_id', p_segment_id::TEXT, FALSE);

    IF p_actor_id IS NULL THEN
        PERFORM set_config('app.actor_id', NULL, FALSE);
    ELSE
        IF NOT EXISTS (
            SELECT 1 FROM app.actors
            WHERE actor_id = p_actor_id
              AND segment_id = p_segment_id
        ) THEN
            RAISE EXCEPTION 'Актёр % не принадлежит сегменту %', p_actor_id, p_segment_id;
        END IF;
        PERFORM set_config('app.actor_id', p_actor_id::TEXT, FALSE);
    END IF;
END;
$$;
```

Рисунок 5 – Функция set_session_ctx

Перед установкой контекста функция выполняет проверку прав доступа. Она проверяет, имеет ли текущая роль пользователя право на работу с

запрашиваемым сегментом, используя таблицу соответствий ролей и сегментов. Дополнительно для роли актёра выполняется проверка принадлежности указанного актёра к запрашиваемому сегменту.

После успешной проверки прав доступа функция устанавливает параметры сессии, которые используются политиками RLS для фильтрации данных. Эти параметры действуют в пределах текущей транзакции и автоматически сбрасываются после её завершения.

Код реализации представлен в приложении А.

Безопасные представления

Созданы безопасные представления для работы с неконфиденциальными полями таблиц projects и contracts. Использована опция WITH CHECK OPTION, чтобы гарантировать, что через представление нельзя вставить или обновить строки, которые не будут видны через это же представление. Для таблицы проекты исключены конфиденциальные поля client_contacts и budget (Рисунок 6).

```
CREATE VIEW app.projects_public_view AS
SELECT
    project_id,
    studio_id,
    segment_id,
    title,
    original_language,
    description,
    start_date,
    end_date,
    status,
    created_at
FROM app.projects
WITH CHECK OPTION;
```

Рисунок 6 – Создание безопасного представления

Созданы представления с опцией SECURITY BARRIER, чтобы предотвратить утечку данных через условия HAVING или подзапросы. Представления возвращают только агрегированные данные, а также исключена возможность получения деталей через побочные каналы (Рисунок 7).

```
CREATE VIEW app.projects_statistics_view
WITH (security_barrier = true) AS
SELECT
    segment_id,
    status,
    COUNT(*) as project_count,
    COUNT(DISTINCT studio_id) as studios_count,
    MIN(start_date) as earliest_start,
    MAX(end_date) as latest_end
FROM app.projects
GROUP BY segment_id, status;
```

Рисунок 7 – Создание view с security barrier

Код для создания безопасных представлений и view с security barrier представлен в приложении А.

История изменений в audit.row_change_log

Для обеспечения полной отслеживаемости всех операций с важными данными была разработана система аудита. Основой системы стала таблица audit.row_change_log, которая фиксирует каждое изменение в защищаемых таблицах базы данных.

Таблица спроектирована для хранения исчерпывающей информации о каждой операции модификации данных, включая метаданные о времени выполнения, инициаторе изменения и технических параметрах запроса (Рисунок 8).

	log_id [PK] bigint	change_time timestamp with time zone	changed_by character varying (100)	table_schema character varying (100)	table_name character varying (100)	operation_type character varying (10)
1	1	2025-11-26 12:52:56.488696+...	app_owner	app	projects	UPDATE
2	2	2025-11-26 12:52:56.488696+...	app_owner	app	projects	UPDATE
3	3	2025-11-26 12:53:35.814342+...	app_owner	app	projects	UPDATE
4	4	2025-11-26 12:53:35.814342+...	app_owner	app	projects	UPDATE

Рисунок 8 – Таблица audit.row_change_log

Код реализации истории изменений в audit.row_change_log представлен в приложении А.

Анализ производительности

Было проведено сравнение производительности типовых запросов до и после внедрения политик RLS. Для анализа использовались операционные запросы фильтрации проектов по сегменту и статусу, JOIN-запросы между контрактами и актёрами, а также агрегирующие запросы по сессиям записи. Измерялось время выполнения, время планирования и количество прочитанных буферов для каждого сценария (Рисунок 9).

Запрос	RLS	Время выполнения (мс)	Время планирования (мс)	Буферов прочитано	Буферов из кэша
TEST1: Projects by segment and status	НЕТ	0.013	0.117	0	86
TEST1: Projects by segment and status	ДА	0.007	0.029	0	6
TEST2: Contracts with JOIN to actors	НЕТ	0.033	0.389	0	231
TEST2: Contracts with JOIN to actors	ДА	0.018	0.062	0	23
TEST3: Aggregate query on recording_sessions	НЕТ	0.024	0.176	0	102
TEST3: Aggregate query on recording_sessions	ДА	0.021	0.040	0	10

(6 rows)

Рисунок 9 – Анализ производительности

Создание составного индекса (segment_id, status) позволило ускорить запросы с RLS. Код реализации представлен в приложении А.

Автоматическое резервное копирование audit-логов

В таблице audit.row_change_log есть записи, которые старше 30 дней (Рисунок 10).

log_id [PK] bigint	change_time timestamp with time zone	changed_by character varying (100)	table_schema character varying (100)	table_name character varying (100)	operation_type character varying (10)	record_id integer
30	2025-10-23 17:29:48.075112+...	demo_user	app	projects	UPDATE	600
31	2025-10-18 17:29:48.075112+...	demo_user	app	contracts	INSERT	601
32	2025-10-13 17:29:48.075112+...	demo_user	app	actors	DELETE	602
33	2025-10-08 17:29:48.075112+...	demo_user	app	projects	UPDATE	603
34	2025-10-03 17:29:48.075112+...	demo_user	app	recording_sessions	UPDATE	604

Рисунок 10 – Таблица audit.row_change_log

Вызываем функцию, которая переносит старые записи в архивную таблицу audit.row_change_log_archive и удаляет их из основной (Рисунок 11).

```
1  SELECT * FROM audit.backup_audit_logs(30);
```

Data Output | Messages | Notifications

Showing rows: 1 to 1

	records_moved bigint	oldest_record_date timestamp with time zone	newest_record_date timestamp with time zone	backup_time timestamp with time zone
1	9	2025-10-03 17:24:22.494407+...	2025-10-23 17:24:22.494407+...	2025-11-27 17:26:32.012229+...

Рисунок 11 – Вызов функции

Видим, что в таблице audit.row_change_log_archive появились архивные записи старше 30 дней (Рисунок 12).

	log_id [PK] bigint	дата_изменения date	пользователь character varying (100)	таблица text	операция character varying (10)	время_архивирования timestamp without time zo
1	25	2025-10-23	demo_user	app.projects	UPDATE	2025-11-27 17:26:32
2	21	2025-10-23	demo_user	app.projects	UPDATE	2025-11-27 17:26:32
3	19	2025-10-23	demo_user	app.projects	UPDATE	2025-11-27 17:20:55
4	17	2025-10-23	demo_user	app.projects	UPDATE	2025-11-27 17:19:10
5	13	2025-10-23	test_user	app.projects	UPDATE	2025-11-27 17:18:13
6	26	2025-10-18	demo_user	app.contracts	INSERT	2025-11-27 17:26:32
7	22	2025-10-18	demo_user	app.contracts	INSERT	2025-11-27 17:26:32
8	20	2025-10-18	demo_user	app.contracts	INSERT	2025-11-27 17:20:55
9	18	2025-10-18	demo_user	app.contracts	INSERT	2025-11-27 17:19:10
10	14	2025-10-18	test_user	app.contracts	INSERT	2025-11-27 17:18:13

Рисунок 12 – Таблица архивирования

Теперь убедимся, что в основной таблице audit.row_change_log старые записи удалились (Рисунок 13).

```
1  select * from audit.row_change_log;
```

Data Output | Messages | Notifications

Showing rows: 1 to 12 | Page No: 1 of 1

	log_id [PK] bigint	change_time timestamp with time zone	changed_by character varying (100)	table_schema character varying (100)	table_name character varying (100)	operation_type character varying (10)
1	1	2025-11-26 12:52:56.488696+...	app_owner	app	projects	UPDATE
2	2	2025-11-26 12:52:56.488696+...	app_owner	app	projects	UPDATE
3	3	2025-11-26 12:53:35.814342+...	app_owner	app	projects	UPDATE
4	4	2025-11-26 12:53:35.814342+...	app_owner	app	projects	UPDATE
5	5	2025-11-27 07:47:07.740139+...	app_owner	app	projects	UPDATE
6	6	2025-11-27 07:47:07.740139+...	app_owner	app	projects	UPDATE

Рисунок 13 - Таблица audit.row_change_log

Из рисунка 13 видим, что функция сработала успешно. Код реализации представлен в приложении А.

Система временных привилегий

Была создана таблица аудита audit.temp_access_log для хранения истории запросов (Рисунок 14).

```
1  select * from audit.temp_access_log;
```

Data Output | Messages | Notifications

Showing rows: 1 to 2 | Page No: 1 of 1

	request_id [PK] bigint	request_time timestamp with time zone	caller_role character varying (100)	caller_username character varying (100)	operation character varying (100)	expires_at timestamp with time zone
1	1	2025-12-19 08:32:27.737017+...	app_writer	postgres	delete_contract	2025-12-19 08:37:27.737017+...
2	2	2025-12-19 08:32:27.772549+...	app_writer	postgres	delete_contract	2025-12-19 08:31:27.772549+...

Рисунок 14 – Таблица audit.temp_access_log

Была создана функция запроса app.request_temp_privilege – проверяет разрешённую операцию в справочнике audit.allowed_temp_operations, записывает запрос в лог, устанавливает время истечения (Рисунок 15).

```
1  select * from audit.allowed_temp_operations;
```

Data Output | Messages | Notifications

Showing rows: 1 to 5 | Page No: 1 of 1

	operation_id [PK] integer	operation_name character varying (100)	role_name character varying (100)	description text	is_active boolean	created_at timestamp with time zone
1	1	delete_contract	app_writer	Удаление контракта	true	2025-12-19 08:32:27.572295+...
2	2	delete_actor	app_writer	Удаление актёра	true	2025-12-19 08:32:27.572295+...
3	3	delete_project	app_writer	Удаление проекта	true	2025-12-19 08:32:27.572295+...
4	4	delete_recording_sessi...	app_writer	Удаление сессии запи...	true	2025-12-19 08:32:27.572295+...
5	5	delete_audio_archive	app_writer	Удаление аудиоархива	true	2025-12-19 08:32:27.572295+...

Рисунок 15 – Таблица справочник app.request_temp_privilege

Также создана функция проверки audit.check_temp_privilege – проверяет наличие активной (не истекшей) привилегии для текущего пользователя, автоматически деактивирует истёкшие.

Принцип работы: пользователь вызывает request_temp_privilege (delete_contract, 5). Система проверяет права и создаёт запись с expires_at = NOW() + 5 min. При вызове safe_delete_contract() функция проверяет активную привилегию через check_temp_privilege(). После истечения 5 минут

check_temp_privilege() автоматически помечает привилегию как неактивную и блокирует операцию.

Тест 1: Запрос привилегии на удаление контракта на 5 минут. Проверяет корректность выдачи и записи в лог (Рисунок 16).

```
NOTICE:  === ТЕСТ 1: Запрос временной привилегии ===
DO
SET
DO
NOTICE:  Функция app.set_session_ctx не найдена (продолжаем без контекста)
DO
NOTICE:  ✅ ТЕСТ 1.1 ПРОЙДЕН: Привилегия запрошена успешно
NOTICE:     Request ID: 1
NOTICE:     Операция: delete_contract
NOTICE:     Истекает: 2025-12-19T08:37:27.737017+00:00
NOTICE:     Длительность: 5 минут
DO
NOTICE:  ✅ ТЕСТ 1.2 ПРОЙДЕН: Запись в audit.temp_access_log создана
NOTICE:     Request ID: 1
NOTICE:     Caller: postgres
NOTICE:     Operation: delete_contract
NOTICE:     Expires at: 2025-12-19 08:37:27.737017+00
DO
DO
```

Рисунок 16 – Тест 1

Тест 2: Удаление контракта с активной привилегией. Проверяет, что система разрешает операцию при наличии прав (Рисунок 17).

```
NOTICE:  === ТЕСТ 2: Выполнение операции с активной привилегией ===
NOTICE:  ✅ Используем существующий контракт без сессий: ID = 11
DO
NOTICE:  ✅ ТЕСТ 2.2 ПРОЙДЕН: Контракт успешно удалён с активной привилегией
DO
```

Рисунок 17 – Тест 2

Тест 3: Попытка удаления без предварительного запроса привилегии. Проверяет, что система блокирует операцию (Рисунок 18).

```
NOTICE:  ✅ ТЕСТ 3.1 ПРОЙДЕН: Операция корректно заблокирована без привилегии
NOTICE:     Ошибка: Временная привилегия на удаление контракта не активна или истекла.
ivilege('delete_contract', duration_min)
DO
DO
```

Рисунок 18 – Тест 3

Тест 4: Попытка удаления после принудительного истечения срока привилегии. Проверяет автоматическую блокировку по времени (Рисунок 19).

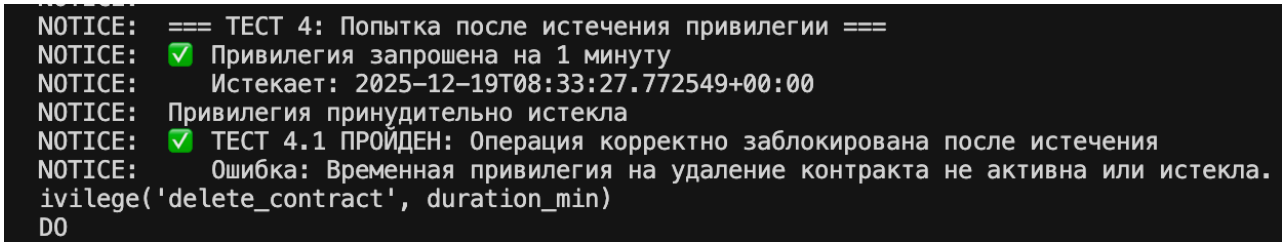


Рисунок 19 – Тест 4

Тест 5: Проверка полноты журналирования в audit.temp_access_log (Рисунок 20).

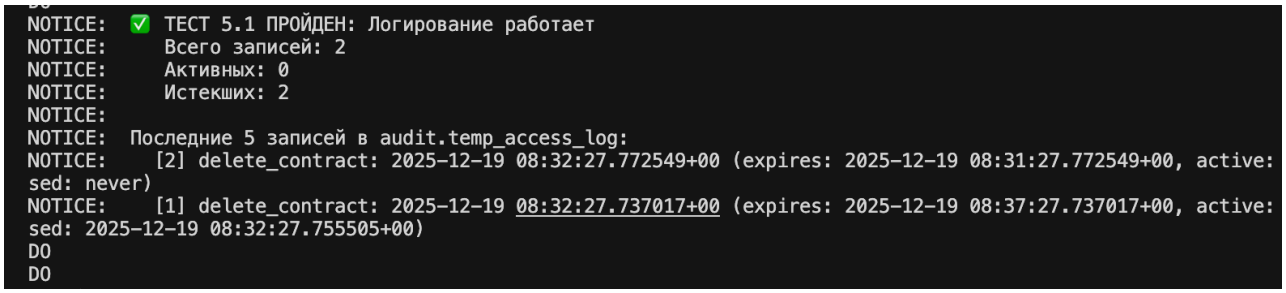


Рисунок 20 – Тест 5

Тест 6: Валидация входных параметров (неразрешённая операция, длительность>60 мин, нулевая длительность) (Рисунок 21).

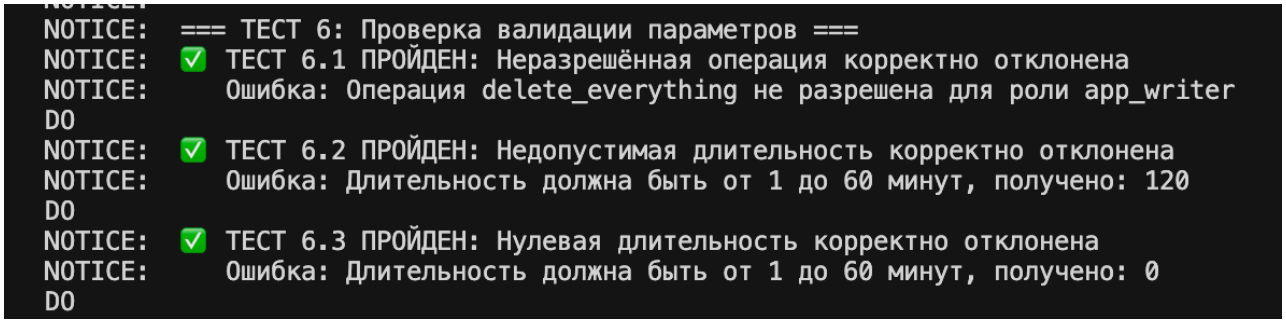


Рисунок 21 – Тест 6

Из рисунок 16-21 видим, что все тесты выполнены успешно. Код реализации представлен в приложении Б.

Вывод

В рамках расчётно-графической работы была успешно разработана и комплексная система безопасности базы данных для студии дубляжа. Основой защиты стал принцип наименьших привилегий, реализованный через ролевую модель, где каждая роль имеет определённый набор прав, соответствующий должностным обязанностям сотрудников – от актёров озвучивания до аудиторов и администраторов безопасности.

Для обеспечения конфиденциальности информации между различными филиалами студии была внедрена система построчной безопасности, которая автоматически ограничивает доступ пользователей только к данным их собственного сегмента, предотвращая несанкционированное пересечение информационных границ.

Главным стало создание системы временных привилегий Just-In-Time Access, которая позволяет предоставлять права на выполнение важных операций, таких как удаление контрактов или записей сессий, исключительно на ограниченный и контролируемый промежуток времени. Данная система функционирует по трёхэтапному принципу: запрос привилегии с указанием длительности, проверка её активности при выполнении операции и автоматический отзыв по истечении срока.

Также были реализованы механизмы полного аудита, включающие логирование всех подключений пользователей, детальное отслеживание изменений данных в таблицах и регистрацию вызовов защищённых функций. Для поддержания производительности системы под нагрузкой проведён анализ выполнения запросов, оптимизированы индексы и политики безопасности, а также настроено автоматическое архивирование устаревших записей журналов, что позволяет сохранять исторические данные без снижения скорости работы оперативных таблиц.

Приложение А

```
-- 01_schemas.sql
-- Создание схем для базы данных студии дубляжа

-- Создаём расширения
CREATE EXTENSION IF NOT EXISTS pgcrypto;

CREATE SCHEMA IF NOT EXISTS app;    -- Основные бизнес-данные
CREATE SCHEMA IF NOT EXISTS ref;    -- Справочники
CREATE SCHEMA IF NOT EXISTS audit;  -- Аудит и логи
CREATE SCHEMA IF NOT EXISTS stg;    -- Временные данные

-- Комментируем схемы
COMMENT ON SCHEMA app IS 'Основные бизнес-данные: контракты, проекты, аудиоархивы';
COMMENT ON SCHEMA ref IS 'Справочники: языки, типы контрактов, студии';
COMMENT ON SCHEMA audit IS 'Аудит и логи безопасности';
COMMENT ON SCHEMA stg IS 'Временная область для обработки и загрузки данных';

-- Создаём роль app_owner для владения объектами
DO $$
BEGIN
   IF NOT EXISTS (SELECT 1 FROM pg_roles WHERE rolname = 'app_owner') THEN
      CREATE ROLE app_owner WITH NOLOGIN NOINHERIT;
   END IF;
END $$;
```

```
-- Отзываем все привилегии у PUBLIC
REVOKE ALL ON SCHEMA app FROM PUBLIC;
REVOKE ALL ON SCHEMA ref FROM PUBLIC;
REVOKE ALL ON SCHEMA audit FROM PUBLIC;
REVOKE ALL ON SCHEMA stg FROM PUBLIC;

-- Устанавливаем владельца схем
ALTER SCHEMA app OWNER TO app_owner;
ALTER SCHEMA ref OWNER TO app_owner;
ALTER SCHEMA audit OWNER TO app_owner;
ALTER SCHEMA stg OWNER TO app_owner;

-- 02_tables.sql
-- Создание таблиц базы данных

-- Устанавливаем поисковый путь
SET search_path TO app, ref, audit, stg, public;

-- Устанавливаем роль для создания объектов
SET ROLE app_owner;

-- Таблицы в схеме ref (справочники)
CREATE TABLE ref.studios (
    studio_id SERIAL PRIMARY KEY,
    studio_name VARCHAR(100) NOT NULL,
    contact_email VARCHAR(100),
    phone VARCHAR(20),
    address TEXT,
    bank_details TEXT,
```

```
    created_at TIMESTAMPTZ DEFAULT NOW(),
    updated_at TIMESTAMPTZ DEFAULT NOW()
);

CREATE TABLE ref.languages (
    language_id SERIAL PRIMARY KEY,
    language_code VARCHAR(10) NOT NULL UNIQUE,
    language_name VARCHAR(50) NOT NULL,
    is_active BOOLEAN DEFAULT true
);

-- Таблицы в схеме app (бизнес-данные)
CREATE TABLE app.actors (
    actor_id SERIAL PRIMARY KEY,
    last_name VARCHAR(100) NOT NULL,
    first_name VARCHAR(100) NOT NULL,
    middle_name VARCHAR(100),
    phone VARCHAR(20),
    email VARCHAR(100),
    passport_data TEXT,
    tax_id VARCHAR(50),
    voice_characteristics TEXT,
    is_active BOOLEAN DEFAULT true,
    created_at TIMESTAMPTZ DEFAULT NOW(),
    updated_at TIMESTAMPTZ DEFAULT NOW()
);

CREATE TABLE app.projects (
    project_id SERIAL PRIMARY KEY,
    studio_id INTEGER NOT NULL REFERENCES ref.studios(studio_id),
```

```
    title VARCHAR(300) NOT NULL,
    original_language VARCHAR(10) NOT NULL,
    description TEXT,
    client_contacts TEXT,
    budget DECIMAL(15,2),
    start_date DATE,
    end_date DATE,
    status VARCHAR(20) DEFAULT 'active',
    created_at TIMESTAMPTZ DEFAULT NOW()
);

CREATE TABLE app.contracts (
    contract_id SERIAL PRIMARY KEY,
    project_id INTEGER NOT NULL REFERENCES app.projects(project_id),
    actor_id INTEGER NOT NULL REFERENCES app.actors(actor_id),
    contract_number VARCHAR(50) NOT NULL UNIQUE,
    role_description TEXT NOT NULL,
    salary_amount DECIMAL(10,2) NOT NULL,
    currency VARCHAR(3) DEFAULT 'USD',
    terms TEXT,
    signed_date DATE,
    end_date DATE,
    status VARCHAR(20) DEFAULT 'active',
    created_at TIMESTAMPTZ DEFAULT NOW()
);

CREATE TABLE app.recording_sessions (
    session_id SERIAL PRIMARY KEY,
    contract_id INTEGER NOT NULL REFERENCES app.contracts(contract_id),
    session_date DATE NOT NULL,
```

```
    session_time TIME NOT NULL,
    studio_location VARCHAR(100) NOT NULL,
    duration_hours DECIMAL(4,2),
    status VARCHAR(20) DEFAULT 'scheduled',
    notes TEXT,
    created_at TIMESTAMPTZ DEFAULT NOW()
);

CREATE TABLE app.audio_archives (
    archive_id SERIAL PRIMARY KEY,
    session_id INTEGER NOT NULL REFERENCES
app.recording_sessions(session_id),
    file_name VARCHAR(255) NOT NULL,
    file_path TEXT NOT NULL,
    file_size BIGINT,
    duration_seconds INTEGER,
    encrypted_audio BYTEA,
    encryption_key_id VARCHAR(100),
    quality_check_passed BOOLEAN DEFAULT false,
    upload_date TIMESTAMPTZ DEFAULT NOW(),
    created_at TIMESTAMPTZ DEFAULT NOW()
);

CREATE TABLE app.voice_roles (
    role_id SERIAL PRIMARY KEY,
    project_id INTEGER NOT NULL REFERENCES app.projects(project_id),
    actor_id INTEGER NOT NULL REFERENCES app.actors(actor_id),
    language_id INTEGER NOT NULL REFERENCES ref.languages(language_id),
    character_name VARCHAR(100) NOT NULL,
    role_type VARCHAR(50) NOT NULL,
```

```
    difficulty_level INTEGER CHECK (difficulty_level BETWEEN 1 AND 5),
    created_at TIMESTAMPTZ DEFAULT NOW(),
    UNIQUE(project_id, actor_id, language_id, character_name)
);

-- Таблицы в схеме audit (логирование)
CREATE TABLE audit.login_log (
    log_id BIGSERIAL PRIMARY KEY,
    login_time TIMESTAMPTZ DEFAULT NOW() NOT NULL,
    username VARCHAR(100) NOT NULL,
    client_ip INET,
    connection_method VARCHAR(50),
    is_successful BOOLEAN DEFAULT true,
    failure_reason TEXT
);

CREATE TABLE audit.data_access_log (
    access_id BIGSERIAL PRIMARY KEY,
    access_time TIMESTAMPTZ DEFAULT NOW() NOT NULL,
    username VARCHAR(100) NOT NULL,
    table_name VARCHAR(100) NOT NULL,
    operation_type VARCHAR(10) NOT NULL,
    record_id INTEGER,
    old_values JSONB,
    new_values JSONB,
    client_ip INET
);

-- Таблицы в схеме stg (временные/обслуживающие данные)
CREATE TABLE stg.import_log (
```

```
    import_id SERIAL PRIMARY KEY,
    import_date TIMESTAMPTZ DEFAULT NOW() NOT NULL,
    source_file VARCHAR(255) NOT NULL,
    records_processed INTEGER DEFAULT 0,
    records_success INTEGER DEFAULT 0,
    records_failed INTEGER DEFAULT 0,
    status VARCHAR(20) DEFAULT 'running',
    error_message TEXT,
    created_by VARCHAR(100) NOT NULL
);

CREATE TABLE stg.temp_actor_data (
    temp_id SERIAL PRIMARY KEY,
    last_name VARCHAR(100),
    first_name VARCHAR(100),
    middle_name VARCHAR(100),
    phone VARCHAR(20),
    email VARCHAR(100),
    voice_characteristics TEXT,
    import_batch_id INTEGER,
    validation_status VARCHAR(20) DEFAULT 'pending',
    validation_errors TEXT,
    created_at TIMESTAMPTZ DEFAULT NOW()
);

-- Создаём индексы для производительности
CREATE INDEX idx_contracts_project_id ON app.contracts(project_id);
CREATE INDEX idx_contracts_actor_id ON app.contracts(actor_id);
CREATE INDEX idx_sessions_contract_id ON
app.recording_sessions(contract_id);
```

```
CREATE INDEX idx_audio_session_id ON app.audio_archives(session_id);
CREATE INDEX idx_voice_roles_project_id ON app.voice_roles(project_id);
CREATE INDEX idx_voice_roles_actor_id ON app.voice_roles(actor_id);
CREATE INDEX idx_voice_roles_language_id ON app.voice_roles(language_id);
CREATE INDEX idx_actors_active ON app.actors(is_active) WHERE is_active =
true;
CREATE INDEX idx_projects_status ON app.projects(status);
CREATE INDEX idx_contracts_status ON app.contracts(status);
CREATE INDEX idx_sessions_date ON app.recording_sessions(session_date);
CREATE INDEX idx_login_log_time ON audit.login_log(login_time);
CREATE INDEX idx_login_log_user ON audit.login_log(username);

-- Комментарии к таблицам и колонкам
COMMENT ON TABLE ref.studios IS 'Студии дубляжа';
COMMENT ON COLUMN ref.studios.bank_details IS 'Confidential: Банковские
реквизиты';
COMMENT ON TABLE ref.languages IS 'Языки для дубляжа';
COMMENT ON TABLE app.actors IS 'Актёры озвучивания';
COMMENT ON COLUMN app.actors.last_name IS 'Internal: Фамилия актёра';
COMMENT ON COLUMN app.actors.first_name IS 'Internal: Имя актёра';
COMMENT ON COLUMN app.actors.middle_name IS 'Internal: Отчество актёра';
COMMENT ON COLUMN app.actors.passport_data IS 'Restricted: Паспортные
данные';
COMMENT ON COLUMN app.actors.tax_id IS 'Confidential: ИНН/налоговый
номер';
COMMENT ON TABLE app.projects IS 'Проекты дубляжа (фильмы, сериалы)';
COMMENT ON COLUMN app.projects.client_contacts IS 'Internal: Контакты
клиента';
COMMENT ON TABLE app.contracts IS 'Контракты с актёрами';
```

```
COMMENT ON COLUMN app.contracts.salary_amount IS 'Confidential: Сумма контракта';
COMMENT ON COLUMN app.contracts.terms IS 'Confidential: Условия контракта';
COMMENT ON TABLE app.recording_sessions IS 'Сессии записи дубляжа';
COMMENT ON TABLE app.audio_archives IS 'Архив аудиофайлов';
COMMENT ON TABLE app.voice_roles IS 'Роли озвучивания';
COMMENT ON TABLE audit.login_log IS 'Логи подключений пользователей';
COMMENT ON TABLE audit.data_access_log IS 'Логи доступа к данным';

-- Сбрасываем роль
RESET ROLE;

-- 03_ref_data.sql
-- Наполнение справочников базовыми данными

-- Языки
INSERT INTO ref.languages (language_code, language_name) VALUES
('ru', 'Русский'),
('en', 'Английский'),
('de', 'Немецкий'),
('fr', 'Французский'),
('es', 'Испанский'),
('it', 'Итальянский'),
('ja', 'Японский'),
('ko', 'Корейский');

-- Студии
INSERT INTO ref.studios (studio_name, contact_email, phone, address) VALUES
```

('ЗвукЗапись', 'studio1@example.com', '+7-495-111-2233', 'Москва, ул. Студийная, 1'),
('ГолосДубляжа', 'studio2@example.com', '+7-812-222-3344', 'Санкт-Петербург, пр. Записной, 15'),
('АудиоМир', 'studio3@example.com', '+7-383-333-4455', 'Новосибирск, ул. Микрофонная, 7'),
('ВолнаЗвука', 'studio4@example.com', '+7-343-444-5566', 'Екатеринбург, ул. Акустическая, 42'),
('Синхрон', 'studio5@example.com', '+7-401-555-6677', 'Калининград, пр. Голосовой, 88'),
('РечевойЦентр', 'studio6@example.com', '+7-391-666-7788', 'Красноярск, ул. Голосовая, 25'),
('ЗвуковыеВолны', 'studio7@example.com', '+7-423-777-8899', 'Владивосток, пр. Звуковой, 10'),
('Артикуляция', 'studio8@example.com', '+7-846-888-9900', 'Самара, ул. Записная, 33'),
('ГолосМастер', 'studio9@example.com', '+7-831-999-0011', 'Нижний Новгород, пр. Студийный, 50'),
('АудиоПрофи', 'studio10@example.com', '+7-351-000-1122', 'Челябинск, ул. Микрофонная, 18');

-- 04_roles.sql
-- Создание системы ролей с разделением обязанностей

-- Базовые роли доступа (с проверкой существования)

```
DO $$
BEGIN
   IF NOT EXISTS (SELECT 1 FROM pg_roles WHERE rolname = 'app_reader')
THEN
```

```
    CREATE ROLE app_reader WITH NOLOGIN NOINHERIT;
  END IF;

  IF NOT EXISTS (SELECT 1 FROM pg_roles WHERE rolname = 'app_writer')
THEN
    CREATE ROLE app_writer WITH NOLOGIN NOINHERIT;
  END IF;

  IF NOT EXISTS (SELECT 1 FROM pg_roles WHERE rolname = 'auditor')
THEN
    CREATE ROLE auditor WITH NOLOGIN NOINHERIT;
  END IF;
END $$;

-- Роли разделения обязанностей (с проверкой существования)
DO $$
BEGIN
  IF NOT EXISTS (SELECT 1 FROM pg_roles WHERE rolname = 'ddl_admin')
THEN
    CREATE ROLE ddl_admin WITH NOLOGIN NOINHERIT;
  END IF;

  IF NOT EXISTS (SELECT 1 FROM pg_roles WHERE rolname = 'dml_admin')
THEN
    CREATE ROLE dml_admin WITH NOLOGIN NOINHERIT;
  END IF;

  IF NOT EXISTS (SELECT 1 FROM pg_roles WHERE rolname =
'security_admin') THEN
    CREATE ROLE security_admin WITH NOLOGIN NOINHERIT;
```

```
    END IF;
END $$;

-- Логин-роли (пользователи)
DO $$
BEGIN
    -- Удаляем существующих пользователей если нужно пересоздать
    IF EXISTS (SELECT 1 FROM pg_roles WHERE rolname = 'studio_manager')
THEN
        RAISE NOTICE 'Роль studio_manager уже существует, переиспользуем';
    ELSE
        CREATE ROLE studio_manager WITH LOGIN PASSWORD
'manager_pass123' NOINHERIT;
    END IF;

    IF EXISTS (SELECT 1 FROM pg_roles WHERE rolname = 'voice_actor') THEN
        RAISE NOTICE 'Роль voice_actor уже существует, переиспользуем';
    ELSE
        CREATE ROLE voice_actor WITH LOGIN PASSWORD 'actor_pass123'
NOINHERIT;
    END IF;

    IF EXISTS (SELECT 1 FROM pg_roles WHERE rolname = 'security_auditor')
THEN
        RAISE NOTICE 'Роль security_auditor уже существует, переиспользуем';
    ELSE
        CREATE ROLE security_auditor WITH LOGIN PASSWORD
'auditor_pass123' NOINHERIT;
    END IF;
```

```
    IF EXISTS (SELECT 1 FROM pg_roles WHERE rolname = 'app_owner_user')
THEN
        RAISE NOTICE 'Роль app_owner_user уже существует, переиспользуем';
    ELSE
        CREATE ROLE app_owner_user WITH LOGIN PASSWORD
'owner_pass123' NOINHERIT;
    END IF;
END $$;

-- Комментарии к ролям (обновляем если изменились)
COMMENT ON ROLE app_owner IS 'Владелец приложения: полный доступ к
бизнес-данным';
COMMENT ON ROLE app_reader IS 'Чтение бизнес-данных: просмотр
проектов, контрактов';
COMMENT ON ROLE app_writer IS 'Запись бизнес-данных:
добавление/изменение контрактов, сессий';
COMMENT ON ROLE auditor IS 'Аудитор: доступ к логам безопасности только
на чтение';
COMMENT ON ROLE ddl_admin IS 'Администратор DDL: создание/изменение
таблиц, индексов';
COMMENT ON ROLE dml_admin IS 'Администратор DML: массовые операции
INSERT/UPDATE/DELETE';
COMMENT ON ROLE security_admin IS 'Администратор безопасности: RLS,
политики, шифрование';
COMMENT ON ROLE studio_manager IS 'Менеджер студии: работает с
контрактами и расписаниями';
COMMENT ON ROLE voice_actor IS 'Актёр: просматривает свои сессии и
роли';
COMMENT ON ROLE security_auditor IS 'Аудитор безопасности: проверяет
логи доступа';
```

```
COMMENT ON ROLE app_owner_user IS 'Пользователь-владелец: полный доступ к системе';

-- Назначаем роли пользователям
GRANT app_writer TO studio_manager;
GRANT app_reader TO voice_actor;
GRANT auditor TO security_auditor;
GRANT app_owner TO app_owner_user;

-- Отзываем все привилегии у PUBLIC
REVOKE ALL ON DATABASE postgres FROM PUBLIC;

-- Уведомление о завершении
DO $$
BEGIN
    RAISE NOTICE 'Роли успешно созданы/обновлены';
    RAISE NOTICE 'Логин-роли: studio_manager, voice_actor, security_auditor, app_owner_user';
    RAISE NOTICE 'Пароли: manager_pass123, actor_pass123, auditor_pass123, owner_pass123';
END $$;

-- 05_privileges.sql
-- Настройка привилегий для ролей

-- Привилегии на уровне схем для ролей
GRANT USAGE ON SCHEMA app, ref, stg TO app_owner;
GRANT ALL ON SCHEMA app, ref, stg TO app_owner;

GRANT USAGE ON SCHEMA app, ref TO app_reader;
```

```
GRANT USAGE ON SCHEMA app, ref TO app_writer;
GRANT USAGE ON SCHEMA audit TO auditor;

GRANT USAGE ON SCHEMA app, ref, audit, stg TO ddl_admin, dml_admin,
security_admin;
GRANT CREATE ON SCHEMA app, ref, audit, stg TO ddl_admin;

-- ЯВНЫЕ ПРАВА ДЛЯ КОНКРЕТНЫХ ПОЛЬЗОВАТЕЛЕЙ
GRANT USAGE ON SCHEMA app, ref TO voice_actor, studio_manager;
GRANT USAGE ON SCHEMA audit TO security_auditor;
GRANT USAGE ON SCHEMA app, ref, audit, stg TO app_owner_user;

-- Привилегии на таблицы для ролей
GRANT SELECT ON ALL TABLES IN SCHEMA app TO app_reader;
GRANT SELECT ON ALL TABLES IN SCHEMA ref TO app_reader;

GRANT SELECT, INSERT, UPDATE ON ALL TABLES IN SCHEMA app TO
app_writer;
GRANT SELECT, INSERT, UPDATE ON ALL TABLES IN SCHEMA ref TO
app_writer;

-- Ограничиваем доступ к чувствительным колонкам для app_reader
-- Убираем SELECT на salary_amount и terms из contracts
REVOKE SELECT ON app.contracts FROM app_reader;
GRANT SELECT (contract_id, project_id, actor_id, contract_number,
role_description, currency, signed_date, end_date, status, created_at) ON
app.contracts TO app_reader;

-- ЯВНЫЕ ПРАВА НА ТАБЛИЦЫ ДЛЯ ПОЛЬЗОВАТЕЛЕЙ
GRANT SELECT ON ALL TABLES IN SCHEMA app TO voice_actor;
```

```
GRANT SELECT ON ALL TABLES IN SCHEMA ref TO voice_actor;

GRANT SELECT, INSERT, UPDATE ON ALL TABLES IN SCHEMA app TO
studio_manager;
GRANT SELECT, INSERT, UPDATE ON ALL TABLES IN SCHEMA ref TO
studio_manager;

GRANT SELECT ON ALL TABLES IN SCHEMA audit TO security_auditor;

GRANT ALL ON ALL TABLES IN SCHEMA app, ref, audit, stg TO
app_owner_user;

-- Запрещаем удаление для писателей
REVOKE DELETE ON ALL TABLES IN SCHEMA app FROM app_writer,
studio_manager;
REVOKE DELETE ON ALL TABLES IN SCHEMA ref FROM app_writer,
studio_manager;

-- Запрещаем модификации аудита
REVOKE INSERT, UPDATE, DELETE ON ALL TABLES IN SCHEMA audit
FROM PUBLIC;
REVOKE INSERT, UPDATE, DELETE ON ALL TABLES IN SCHEMA audit
FROM app_owner, app_reader, app_writer, auditor, security_auditor;

-- Привилегии для ddl_admin (может читать для анализа структуры)
GRANT SELECT ON ALL TABLES IN SCHEMA app, ref, audit, stg TO
ddl_admin;

-- Привилегии для dml_admin (массовые операции с данными)
```

```
GRANT SELECT, INSERT, UPDATE, DELETE ON ALL TABLES IN SCHEMA
app TO dml_admin;
GRANT SELECT, INSERT, UPDATE, DELETE ON ALL TABLES IN SCHEMA
ref TO dml_admin;
GRANT SELECT, INSERT, UPDATE, DELETE ON ALL TABLES IN SCHEMA
stg TO dml_admin;

-- Привилегии для последовательностей
GRANT USAGE ON ALL SEQUENCES IN SCHEMA app TO app_owner,
app_writer, studio_manager;
GRANT USAGE ON ALL SEQUENCES IN SCHEMA ref TO app_owner,
app_writer, studio_manager;
GRANT USAGE ON ALL SEQUENCES IN SCHEMA app, ref TO ddl_admin,
app_owner_user;
GRANT USAGE ON ALL SEQUENCES IN SCHEMA app, ref TO dml_admin;

-- Уведомление
DO $$
BEGIN
   RAISE NOTICE 'Привилегии успешно применены ко всем ролям и
пользователям';
END $$;

-- 06_default_privs.sql
-- Настройка привилегий по умолчанию для новых объектов

-- Привилегии для app_owner
ALTER DEFAULT PRIVILEGES FOR ROLE app_owner IN SCHEMA app
GRANT SELECT ON TABLES TO app_reader;
```

```
ALTER DEFAULT PRIVILEGES FOR ROLE app_owner IN SCHEMA app
GRANT SELECT, INSERT, UPDATE ON TABLES TO app_writer;

ALTER DEFAULT PRIVILEGES FOR ROLE app_owner IN SCHEMA app
GRANT ALL ON TABLES TO app_owner;

ALTER DEFAULT PRIVILEGES FOR ROLE app_owner IN SCHEMA ref
GRANT SELECT ON TABLES TO app_reader;

ALTER DEFAULT PRIVILEGES FOR ROLE app_owner IN SCHEMA ref
GRANT SELECT, INSERT, UPDATE ON TABLES TO app_writer;

ALTER DEFAULT PRIVILEGES FOR ROLE app_owner IN SCHEMA audit
GRANT SELECT ON TABLES TO auditor;

-- Привилегии для последовательностей
ALTER DEFAULT PRIVILEGES FOR ROLE app_owner IN SCHEMA app
GRANT USAGE ON SEQUENCES TO app_writer;

ALTER DEFAULT PRIVILEGES FOR ROLE app_owner IN SCHEMA ref
GRANT USAGE ON SEQUENCES TO app_writer;

-- Запрещаем PUBLIC доступ к новым объектам
ALTER DEFAULT PRIVILEGES FOR ROLE app_owner IN SCHEMA app
REVOKE ALL ON TABLES FROM PUBLIC;

ALTER DEFAULT PRIVILEGES FOR ROLE app_owner IN SCHEMA ref
REVOKE ALL ON TABLES FROM PUBLIC;

ALTER DEFAULT PRIVILEGES FOR ROLE app_owner IN SCHEMA audit
```

```
REVOKE ALL ON TABLES FROM PUBLIC;

ALTER DEFAULT PRIVILEGES FOR ROLE app_owner IN SCHEMA stg
REVOKE ALL ON TABLES FROM PUBLIC;

-- Уведомление о настройке default privileges
DO $$
BEGIN
   RAISE NOTICE 'Привилегии по умолчанию успешно настроены';
END $$;

-- 07_audit.sql
-- Создание механизма аудита подключений

-- Функция для логирования подключений
CREATE OR REPLACE FUNCTION audit.log_connection()
RETURNS void
LANGUAGE plpgsql
SECURITY DEFINER
SET search_path = audit, public
AS $$
BEGIN
   INSERT INTO audit.login_log (
      username,
      client_ip,
      connection_method,
      is_successful
   ) VALUES (
      current_user::VARCHAR(100),
      inet_client_addr()::INET,
```

```
        'direct',
        true
    );
EXCEPTION
    WHEN OTHERS THEN
        INSERT INTO audit.login_log (
            username,
            client_ip,
            connection_method,
            is_successful,
            failure_reason
        ) VALUES (
            current_user::VARCHAR(100),
            inet_client_addr()::INET,
            'direct',
            false,
            SQLERRM
        );
END;
$$;

-- Роль для логирования подключений (с проверкой существования)
DO $$
BEGIN
    IF NOT EXISTS (SELECT 1 FROM pg_roles WHERE rolname =
'connection_logger') THEN
        CREATE ROLE connection_logger WITH NOLOGIN;
    END IF;
END $$;
```

```
GRANT USAGE ON SCHEMA audit TO connection_logger;
GRANT INSERT ON audit.login_log TO connection_logger;
GRANT EXECUTE ON FUNCTION audit.log_connection TO connection_logger;

-- Назначаем роль конкретным пользователям
GRANT connection_logger TO studio_manager, voice_actor, security_auditor,
db_developer;

-- Функция для ручного логирования сессии
CREATE OR REPLACE FUNCTION public.log_session_start()
RETURNS void
LANGUAGE plpgsql
SECURITY DEFINER
SET search_path = audit, public
AS $$
BEGIN
   PERFORM audit.log_connection();
END;
$$;

-- Даём доступ к функции логирования сессии только нужным ролям
GRANT EXECUTE ON FUNCTION public.log_session_start TO studio_manager,
voice_actor, security_auditor, db_developer;

COMMENT ON FUNCTION audit.log_connection IS 'Функция для логирования
подключений пользователей';
COMMENT ON FUNCTION public.log_session_start IS 'Функция для ручного
логирования старта сессии';

-- 08_test_data.sql
```

-- Наполнение тестовыми данными

-- Актёры

INSERT INTO app.actors (last_name, first_name, middle_name, phone, email, voice_characteristics) VALUES

('Петров', 'Иван', 'Сергеевич', '+7-911-111-1111', 'ivan@example.com', 'Низкий баритон, эмоциональный'),

('Сидорова', 'Мария', 'Александровна', '+7-922-222-2222', 'maria@example.com', 'Сопрано, чистая дикция'),

('Козлов', 'Алексей', NULL, '+7-933-333-3333', 'alex@example.com', 'Тенор, комедийный тембр'),

('Новикова', 'Елена', 'Дмитриевна', '+7-944-444-4444', 'elena@example.com', 'Меццо-сопрано, драматический'),

('Волков', 'Дмитрий', 'Игоревич', '+7-955-555-5555', 'dmitry@example.com', 'Бас, authoritative'),

('Белова', 'Ольга', 'Владимировна', '+7-966-666-6666', 'olga@example.com', 'Сопрано, детские голоса'),

('Орлов', 'Сергей', NULL, '+7-977-777-7777', 'sergey@example.com', 'Баритон, лирический'),

('Ковалёва', 'Анна', 'Петровна', '+7-988-888-8888', 'anna@example.com', 'Альт, глубокий тембр'),

('Морозов', 'Павел', 'Андреевич', '+7-999-999-9999', 'pavel@example.com', 'Тенор, героический'),

('Павлова', 'Наталья', NULL, '+7-900-000-0000', 'natalia@example.com', 'Сопрано, нежный голос');

-- Проекты

INSERT INTO app.projects (studio_id, title, original_language, description, client_contacts, budget, start_date, end_date) VALUES

(1, 'Звёздные войны: Новый рассвет', 'en', 'Фантастический блокбастер', 'disney@example.com', 500000.00, '2024-01-01', '2024-06-30'),
(2, 'Магия Парижа', 'fr', 'Романтическая комедия', 'frenchfilms@example.com', 250000.00, '2024-02-01', '2024-05-31'),
(1, 'Тайна океана', 'en', 'Документальный фильм', 'natgeo@example.com', 150000.00, '2024-03-01', '2024-04-30'),
(3, 'Берлинские тени', 'de', 'Детективный триллер', 'germanstudio@example.com', 300000.00, '2024-01-15', '2024-07-31'),
(2, 'Римские каникулы', 'it', 'Приключенческий фильм', 'italiancinema@example.com', 200000.00, '2024-02-15', '2024-06-15'),
(4, 'Мадридские ночи', 'es', 'Романтическая драма', 'spanishfilms@example.com', 180000.00, '2024-03-15', '2024-08-31'),
(3, 'Токио: Город огней', 'ja', 'Научно-фантастический фильм', 'japanstudio@example.com', 420000.00, '2024-04-01', '2024-09-30'),
(5, 'Амстердамские каналы', 'nl', 'Детективная драма', 'dutchcinema@example.com', 220000.00, '2024-03-20', '2024-07-20'),
(4, 'Лондонский туман', 'en', 'Мистический триллер', 'britishfilms@example.com', 350000.00, '2024-04-10', '2024-10-31'),
(5, 'Варшавская весна', 'pl', 'Историческая драма', 'polishstudio@example.com', 280000.00, '2024-04-15', '2024-08-15');

-- Контракты
INSERT INTO app.contracts (project_id, actor_id, contract_number, role_description, salary_amount, currency, signed_date) VALUES
(1, 1, 'CNTR-2024-001', 'Главный герой - Люк Скайуокер', 50000.00, 'USD', '2024-01-15'),
(1, 2, 'CNTR-2024-002', 'Принцесса Лея', 45000.00, 'USD', '2024-01-16'),
(1, 3, 'CNTR-2024-003', 'Злодей - Дарт Вейдер', 40000.00, 'USD', '2024-01-17'),
(2, 4, 'CNTR-2024-004', 'Главная героиня - Мари', 30000.00, 'EUR', '2024-02-01'),
(2, 5, 'CNTR-2024-005', 'Главный герой - Пьер', 32000.00, 'EUR', '2024-02-02'),

(3, 6, 'CNTR-2024-006', 'Рассказчик', 20000.00, 'USD', '2024-03-01'),
(4, 7, 'CNTR-2024-007', 'Детектив Мюллер', 35000.00, 'EUR', '2024-01-20'),
(4, 8, 'CNTR-2024-008', 'Подозреваемая Анна', 28000.00, 'EUR', '2024-01-22'),
(5, 9, 'CNTR-2024-009', 'Турист Марко', 25000.00, 'EUR', '2024-02-20'),
(5, 10, 'CNTR-2024-010', 'Гид София', 23000.00, 'EUR', '2024-02-21');

-- Сессии записи
INSERT INTO app.recording_sessions (contract_id, session_date, session_time, studio_location, duration_hours) VALUES
(1, '2024-03-01', '10:00:00', 'Студия А, Москва', 4.0),
(2, '2024-03-01', '14:00:00', 'Студия Б, Москва', 3.5),
(3, '2024-03-02', '11:00:00', 'Студия В, СПб', 5.0),
(4, '2024-03-03', '09:00:00', 'Студия Г, Москва', 4.5),
(5, '2024-03-03', '13:00:00', 'Студия Д, Москва', 3.0),
(6, '2024-03-04', '10:00:00', 'Студия А, Москва', 6.0),
(7, '2024-03-05', '15:00:00', 'Студия Е, Новосибирск', 4.0),
(8, '2024-03-05', '10:00:00', 'Студия Е, Новосибирск', 3.5),
(9, '2024-03-06', '11:00:00', 'Студия Ж, СПб', 4.0),
(10, '2024-03-06', '16:00:00', 'Студия Ж, СПб', 3.0);

-- Роли озвучивания
INSERT INTO app.voice_roles (project_id, actor_id, language_id, character_name, role_type, difficulty_level) VALUES
(1, 1, 1, 'Люк Скайуокер', 'lead', 5),
(1, 2, 1, 'Принцесса Лея', 'lead', 5),
(1, 3, 1, 'Дарт Вейдер', 'villain', 4),
(2, 4, 1, 'Мари', 'lead', 4),
(2, 5, 1, 'Пьер', 'lead', 4),
(3, 6, 1, 'Рассказчик', 'narrator', 3),
(4, 7, 1, 'Детектив Мюллер', 'lead', 4),

```
(4, 8, 1, 'Анна', 'supporting', 3),
(5, 9, 1, 'Марко', 'lead', 3),
(5, 10, 1, 'София', 'supporting', 3);

-- Аудиоархивы
INSERT INTO app.audio_archives (session_id, file_name, file_path, file_size,
duration_seconds, encrypted_audio, encryption_key_id, quality_check_passed)
VALUES
(1, 'luke_skywalker_scene1.wav', '/audio/starwars/luke_scene1.wav', 15728640, 300,
pgp_sym_encrypt('encrypted_audio_data_1', 'encryption_key_123'), 'key_001', true),
(1, 'luke_skywalker_scene2.wav', '/audio/starwars/luke_scene2.wav', 20971520, 400,
pgp_sym_encrypt('encrypted_audio_data_2', 'encryption_key_123'), 'key_002', true),
(2, 'leia_organa_scene1.wav', '/audio/starwars/leia_scene1.wav', 12582912, 250,
pgp_sym_encrypt('encrypted_audio_data_3', 'encryption_key_123'), 'key_003', true),
(3, 'darth_vader_scene1.wav', '/audio/starwars/vader_scene1.wav', 18874368, 350,
pgp_sym_encrypt('encrypted_audio_data_4', 'encryption_key_123'), 'key_004',
false),
(4, 'marie_paris_scene1.wav', '/audio/paris/marie_scene1.wav', 10485760, 200,
pgp_sym_encrypt('encrypted_audio_data_5', 'encryption_key_123'), 'key_005', true),
(5, 'pierre_paris_scene1.wav', '/audio/paris/pierre_scene1.wav', 11534336, 220,
pgp_sym_encrypt('encrypted_audio_data_6', 'encryption_key_123'), 'key_006', true),
(6, 'narrator_ocean_scene1.wav', '/audio/ocean/narrator_scene1.wav', 25165824, 480,
pgp_sym_encrypt('encrypted_audio_data_7', 'encryption_key_123'), 'key_007', true),
(7, 'muller_berlin_scene1.wav', '/audio/berlin/muller_scene1.wav', 14680064, 280,
pgp_sym_encrypt('encrypted_audio_data_8', 'encryption_key_123'), 'key_008', true),
(8, 'anna_berlin_scene1.wav', '/audio/berlin/anna_scene1.wav', 8388608, 160,
pgp_sym_encrypt('encrypted_audio_data_9', 'encryption_key_123'), 'key_009',
false),
```

(9, 'marco_rome_scene1.wav', '/audio/rome/marco_scene1.wav', 10485760, 200, pgp_sym_encrypt('encrypted_audio_data_10', 'encryption_key_123'), 'key_010', true),
(10, 'sofia_rome_scene1.wav', '/audio/rome/sofia_scene1.wav', 9437184, 180, pgp_sym_encrypt('encrypted_audio_data_11', 'encryption_key_123'), 'key_011', true);

-- Данные для stg.import_log (логи импорта)
INSERT INTO stg.import_log (source_file, records_processed, records_success, records_failed, status, created_by) VALUES
('actors_2024_01.xlsx', 25, 23, 2, 'completed', 'admin'),
('projects_2024_q1.csv', 15, 15, 0, 'completed', 'studio_manager'),
('contracts_march_2024.xlsx', 30, 28, 2, 'completed', 'admin'),
('voice_actors_new.csv', 12, 10, 2, 'failed', 'admin'),
('audio_archives_batch_001.zip', 50, 48, 2, 'completed', 'db_developer'),
('studios_update_2024.xlsx', 8, 8, 0, 'completed', 'admin'),
('languages_reference.csv', 15, 15, 0, 'completed', 'admin'),
('sessions_april_2024.xlsx', 20, 18, 2, 'completed', 'studio_manager'),
('temp_actors_validation.csv', 5, 0, 5, 'failed', 'admin'),
('migration_legacy_system.sql', 100, 95, 5, 'completed', 'db_developer');

-- Данные для stg.temp_actor_data (временные данные актёров)
INSERT INTO stg.temp_actor_data (last_name, first_name, middle_name, phone, email, voice_characteristics, import_batch_id, validation_status, validation_errors) VALUES
('Новиков', 'Александр', 'Викторович', '+7-900-111-1111', 'alex.novikov@example.com', 'Тенор, драматический', 1, 'validated', NULL),
('Смирнова', 'Екатерина', 'Олеговна', '+7-900-222-2222', 'katya.smirnova@example.com', 'Сопрано, лирический', 1, 'validated', NULL),

('Козлов', 'Михаил', NULL, '+7-900-333-3333', 'misha.kozlov@example.com',
'Баритон, комедийный', 1, 'pending', NULL),
('Петрова', 'Анна', 'Сергеевна', '+7-900-444-4444', 'anna.petrova@example.com',
'Меццо-сопрано, глубокий', 1, 'validated', NULL),
('Волков', 'Дмитрий', 'Игоревич', '+7-900-555-5555',
'dmitry.volkov@example.com', 'Бас, authoritative', 1, 'failed', 'Неверный формат
телефона'),
('Белова', 'Ольга', 'Владимировна', '+7-900-666-6666', 'olga.belova@example.com',
'Сопрано, детские голоса', 2, 'validated', NULL),
('Орлов', 'Сергей', 'Николаевич', '+7-900-777-7777', 'sergey.orlov@example.com',
'Баритон, лирический', 2, 'pending', NULL),
('Павлова', 'Наталья', NULL, '+7-900-888-8888', 'natalia.pavlova@example.com',
'Сопрано, нежный голос', 2, 'failed', 'Отсутствует email'),
('Морозов', 'Павел', 'Андреевич', '+7-900-999-9999',
'pavel.morozov@example.com', 'Тенор, героический', 2, 'validated', NULL),
('Ковалёва', 'Елена', 'Дмитриевна', '+7-900-000-0000',
'elena.kovaleva@example.com', 'Альт, глубокий тембр', 2, 'pending', NULL);

-- Логируем начальные подключения
SELECT public.log_session_start();

-- 09_rls.sql (исправленная часть)
-- Настройка Row Level Security и политик доступа

-- Включаем RLS на критически важных таблицах
ALTER TABLE app.actors ENABLE ROW LEVEL SECURITY;
ALTER TABLE app.contracts ENABLE ROW LEVEL SECURITY;
ALTER TABLE app.recording_sessions ENABLE ROW LEVEL SECURITY;
ALTER TABLE app.audio_archives ENABLE ROW LEVEL SECURITY;
ALTER TABLE app.projects ENABLE ROW LEVEL SECURITY;

```
-- Принудительно включаем RLS
ALTER TABLE app.actors FORCE ROW LEVEL SECURITY;
ALTER TABLE app.contracts FORCE ROW LEVEL SECURITY;
ALTER TABLE app.recording_sessions FORCE ROW LEVEL SECURITY;
ALTER TABLE app.audio_archives FORCE ROW LEVEL SECURITY;
ALTER TABLE app.projects FORCE ROW LEVEL SECURITY;

-- Удаляем существующие политики если они есть
DROP POLICY IF EXISTS actor_self_select ON app.actors;
DROP POLICY IF EXISTS actor_own_contracts ON app.contracts;
DROP POLICY IF EXISTS actor_own_sessions ON app.recording_sessions;
DROP POLICY IF EXISTS studio_own_projects ON app.projects;
DROP POLICY IF EXISTS hide_salaries ON app.contracts;

-- Политика для таблицы actors - актёры видят только свою запись
CREATE POLICY actor_self_select ON app.actors
    FOR SELECT
    TO app_reader, app_writer, voice_actor
    USING (actor_id = current_setting('app.current_actor_id', true)::INTEGER);

-- Политика для dml_admin - полный доступ к actors
CREATE POLICY dml_admin_actors ON app.actors
    FOR ALL
    TO dml_admin
    USING (true);

-- Политика для contracts - актёры видят только свои контракты
CREATE POLICY actor_own_contracts ON app.contracts
    FOR SELECT
```

```
    TO app_reader, app_writer, voice_actor
    USING (actor_id = current_setting('app.current_actor_id', true)::INTEGER);

-- Политика для dml_admin - полный доступ к contracts
CREATE POLICY dml_admin_contracts ON app.contracts
    FOR ALL
    TO dml_admin
    USING (true);

-- Политика для recording_sessions - через JOIN к contracts
CREATE POLICY actor_own_sessions ON app.recording_sessions
    FOR SELECT
    TO app_reader, app_writer, voice_actor
    USING (EXISTS (
        SELECT 1 FROM app.contracts c
        WHERE c.contract_id = app.recording_sessions.contract_id
        AND c.actor_id = current_setting('app.current_actor_id', true)::INTEGER
    ));

-- Политика для dml_admin - полный доступ к recording_sessions
CREATE POLICY dml_admin_sessions ON app.recording_sessions
    FOR ALL
    TO dml_admin
    USING (true);

-- Функция для определения студии менеджера (пересоздаём)
CREATE OR REPLACE FUNCTION app.get_current_user_studio()
RETURNS INTEGER
LANGUAGE sql
STABLE
```

```
AS $$
    SELECT 1; -- Для демонстрации возвращаем studio_id = 1
$$;

-- Политика для projects - менеджеры видят проекты своей студии
CREATE POLICY studio_own_projects ON app.projects
    FOR ALL
    TO app_writer, app_reader, studio_manager
    USING (studio_id = app.get_current_user_studio());

-- Политика для dml_admin - полный доступ к projects
CREATE POLICY dml_admin_projects ON app.projects
    FOR ALL
    TO dml_admin
    USING (true);

-- Политика для скрытия конфиденциальных зарплат
CREATE POLICY hide_salaries ON app.contracts
    FOR SELECT
    TO app_reader
    USING (true);

-- SECURITY DEFINER функция для безопасного доступа к контрактам (пересоздаём)
CREATE OR REPLACE FUNCTION app.get_contracts_with_masked_salary()
RETURNS TABLE (
    contract_id INTEGER,
    contract_number VARCHAR,
    role_description TEXT,
    salary_range VARCHAR,
```

```
    currency VARCHAR,
    signed_date DATE
)
LANGUAGE sql
SECURITY DEFINER
SET search_path = app, public
AS $$
    SELECT
        contract_id,
        contract_number,
        role_description,
        CASE
            WHEN salary_amount < 10000 THEN 'до 10,000'
            WHEN salary_amount < 50000 THEN '10,000 - 50,000'
            ELSE 'свыше 50,000'
        END as salary_range,
        currency,
        signed_date
    FROM app.contracts;
$$;

-- Даём доступ к функции
GRANT EXECUTE ON FUNCTION app.get_contracts_with_masked_salary TO
app_reader;

COMMENT ON FUNCTION app.get_contracts_with_masked_salary IS
'Безопасный просмотр контрактов с маскировкой зарплат';

-- Политика для dml_admin - полный доступ к audio_archives
CREATE POLICY dml_admin_audio_archives ON app.audio_archives
```

```
    FOR ALL
    TO dml_admin
    USING (true);

-- Уведомление о завершении
DO $$
BEGIN
    RAISE NOTICE 'RLS политики и функции безопасности успешно
настроены';
END $$;

-- 10_audit_triggers_and_tests.sql
-- Триггеры аудита DML операций и тестирование системы безопасности

-- Функция для логирования изменений данных
CREATE OR REPLACE FUNCTION audit.log_data_changes()
RETURNS TRIGGER
LANGUAGE plpgsql
SECURITY DEFINER
SET search_path = audit, public
AS $$
DECLARE
    old_data JSONB;
    new_data JSONB;
    operation_type VARCHAR(10);
BEGIN
    -- Определяем тип операции
    IF TG_OP = 'DELETE' THEN
        operation_type := 'DELETE';
        old_data := to_jsonb(OLD);
```

```
    new_data := NULL;
ELSIF TG_OP = 'UPDATE' THEN
    operation_type := 'UPDATE';
    old_data := to_jsonb(OLD);
    new_data := to_jsonb(NEW);
ELSIF TG_OP = 'INSERT' THEN
    operation_type := 'INSERT';
    old_data := NULL;
    new_data := to_jsonb(NEW);
END IF;

-- Записываем в лог аудита
INSERT INTO audit.data_access_log (
    username,
    table_name,
    operation_type,
    record_id,
    old_values,
    new_values,
    client_ip
) VALUES (
    current_user::VARCHAR(100),
    TG_TABLE_SCHEMA || '.' || TG_TABLE_NAME,
    operation_type,
    CASE
        WHEN TG_OP = 'DELETE' THEN OLD.actor_id
        WHEN TG_OP = 'UPDATE' THEN NEW.actor_id
        WHEN TG_OP = 'INSERT' THEN NEW.actor_id
    END,
    old_data,
```

```
        new_data,
        inet_client_addr()
    );

    -- Возвращаем соответствующий результат
    IF TG_OP = 'DELETE' THEN
        RETURN OLD;
    ELSE
        RETURN NEW;
    END IF;
EXCEPTION
    WHEN OTHERS THEN
        -- В случае ошибки логируем и продолжаем
        INSERT INTO audit.data_access_log (
            username,
            table_name,
            operation_type,
            old_values,
            new_values,
            client_ip
        ) VALUES (
            current_user::VARCHAR(100),
            TG_TABLE_SCHEMA || '.' || TG_TABLE_NAME,
            'ERROR',
            NULL,
            jsonb_build_object('error', SQLERRM),
            inet_client_addr()
        );

        IF TG_OP = 'DELETE' THEN
```

```
        RETURN OLD;
    ELSE
        RETURN NEW;
    END IF;
END;
$$;

-- Создаём триггеры для основных таблиц
CREATE TRIGGER audit_actors_changes
    AFTER INSERT OR UPDATE OR DELETE ON app.actors
    FOR EACH ROW EXECUTE FUNCTION audit.log_data_changes();

CREATE TRIGGER audit_contracts_changes
    AFTER INSERT OR UPDATE OR DELETE ON app.contracts
    FOR EACH ROW EXECUTE FUNCTION audit.log_data_changes();

CREATE TRIGGER audit_projects_changes
    AFTER INSERT OR UPDATE OR DELETE ON app.projects
    FOR EACH ROW EXECUTE FUNCTION audit.log_data_changes();

CREATE TRIGGER audit_recording_sessions_changes
    AFTER INSERT OR UPDATE OR DELETE ON app.recording_sessions
    FOR EACH ROW EXECUTE FUNCTION audit.log_data_changes();

CREATE TRIGGER audit_audio_archives_changes
    AFTER INSERT OR UPDATE OR DELETE ON app.audio_archives
    FOR EACH ROW EXECUTE FUNCTION audit.log_data_changes();

-- Даём права на выполнение функции аудита
```

```
GRANT EXECUTE ON FUNCTION audit.log_data_changes TO app_owner,
app_writer;

-- Комментарии
COMMENT ON FUNCTION audit.log_data_changes IS 'Функция для
автоматического логирования изменений в таблицах';
COMMENT ON TRIGGER audit_actors_changes ON app.actors IS 'Триггер
аудита для таблицы актёров';
COMMENT ON TRIGGER audit_contracts_changes ON app.contracts IS 'Триггер
аудита для таблицы контрактов';
COMMENT ON TRIGGER audit_projects_changes ON app.projects IS 'Триггер
аудита для таблицы проектов';
COMMENT ON TRIGGER audit_recording_sessions_changes ON
app.recording_sessions IS 'Триггер аудита для таблицы сессий записи';
COMMENT ON TRIGGER audit_audio_archives_changes ON app.audio_archives
IS 'Триггер аудита для таблицы аудиоархивов';

-- Автоматическое логирование входов через pgaudit
CREATE OR REPLACE FUNCTION audit.parse_pgaudit_log()
RETURNS void
LANGUAGE plpgsql
SECURITY DEFINER
SET search_path = audit, public
AS $$
DECLARE
    log_line TEXT;
    username_part TEXT;
    ip_part TEXT;
    success_part TEXT;
    parsed_username VARCHAR(100);
```

```
    parsed_ip INET;
    parsed_success BOOLEAN;
BEGIN
    -- Эта функция будет вызываться периодически для парсинга логов pgaudit

    -- Для демонстрации создаём пример записи
    INSERT INTO audit.login_log (
        username,
        client_ip,
        connection_method,
        is_successful,
        failure_reason
    ) VALUES (
        'pgaudit_parser',
        '127.0.0.1',
        'pgaudit_auto',
        true,
        NULL
    );

EXCEPTION
    WHEN OTHERS THEN
        -- В случае ошибки логируем
        INSERT INTO audit.login_log (
            username,
            client_ip,
            connection_method,
            is_successful,
            failure_reason
        ) VALUES (
```

```
            'pgaudit_parser',
            '127.0.0.1',
            'pgaudit_auto',
            false,
            SQLERRM
        );
END;
$$;

-- Создаём функцию для принудительного логирования входа при подключении
CREATE OR REPLACE FUNCTION public.auto_log_connection()
RETURNS void
LANGUAGE plpgsql
SECURITY DEFINER
SET search_path = audit, public
AS $$
BEGIN
    -- Логируем подключение
    PERFORM audit.log_connection();

    -- Также парсим pgaudit логи
    PERFORM audit.parse_pgaudit_log();
END;
$$;

-- Даём доступ к функциям
GRANT EXECUTE ON FUNCTION audit.parse_pgaudit_log TO app_owner,
security_auditor;
GRANT EXECUTE ON FUNCTION public.auto_log_connection TO
studio_manager, voice_actor, security_auditor, db_developer;
```

```
-- Создаём роль для автоматического логирования
DO $$
BEGIN
   IF NOT EXISTS (SELECT 1 FROM pg_roles WHERE rolname = 'auto_logger')
THEN
      CREATE ROLE auto_logger WITH NOLOGIN;
   END IF;
END $$;

GRANT USAGE ON SCHEMA audit TO auto_logger;
GRANT INSERT ON audit.login_log TO auto_logger;
GRANT EXECUTE ON FUNCTION audit.log_connection TO auto_logger;
GRANT EXECUTE ON FUNCTION audit.parse_pgaudit_log TO auto_logger;

-- Назначаем роль пользователям
GRANT auto_logger TO studio_manager, voice_actor, security_auditor,
db_developer;

-- Комментарии
COMMENT ON FUNCTION audit.parse_pgaudit_log IS 'Функция для парсинга
логов pgaudit и записи в audit.login_log';
COMMENT ON FUNCTION public.auto_log_connection IS 'Функция для
автоматического логирования подключений';
COMMENT ON ROLE auto_logger IS 'Роль для автоматического логирования
подключений';

-- Уведомление о завершении
DO $$
BEGIN
```

```
    RAISE NOTICE 'Триггеры аудита DML операций и автоматическое
логирование входов настроены';
END $$;

-- Тестирование системы безопасности (выполнять ПОСЛЕ всех скриптов)

DO $$
BEGIN
    RAISE NOTICE '=== НАЧАЛО ТЕСТИРОВАНИЯ СИСТЕМЫ
БЕЗОПАСНОСТИ ===';
END $$;

-- Проверяем существование тестовых данных
DO $$
DECLARE
    actor_count INTEGER;
    contract_count INTEGER;
BEGIN
    SELECT COUNT(*) INTO actor_count FROM app.actors;
    SELECT COUNT(*) INTO contract_count FROM app.contracts;

    RAISE NOTICE 'Найдено актёров: %, контрактов: %', actor_count,
contract_count;

    IF actor_count = 0 OR contract_count = 0 THEN
        RAISE EXCEPTION 'Отсутствуют тестовые данные! Сначала выполните
скрипт 08_test_data.sql';
    END IF;
END $$;
```

```
-- Проверка от имени superuser
DO $$
BEGIN
    RAISE NOTICE '=== ТЕСТ 1: Проверка от superuser ===';
END $$;

SELECT
    'Superuser видит актёров: ' || COUNT(*)::TEXT as test_result
FROM app.actors;

SELECT
    'Superuser видит контракты: ' || COUNT(*)::TEXT as test_result
FROM app.contracts;

SELECT
    'Superuser видит логи: ' || COUNT(*)::TEXT as test_result
FROM audit.login_log;

-- Тестирование безопасных функций
DO $$
BEGIN
    RAISE NOTICE '=== ТЕСТ 2: Безопасные функции ===';
END $$;

SELECT * FROM app.get_contracts_with_masked_salary() LIMIT 3;

-- Автоматическая проверка системы
DO $$
BEGIN
    RAISE NOTICE '=== АВТОМАТИЧЕСКАЯ ПРОВЕРКА СИСТЕМЫ ===';
```

```
END $$;

-- Проверяем роли
SELECT
    rolname,
    CASE
        WHEN rolname IN ('app_reader', 'app_writer', 'auditor', 'studio_manager',
'voice_actor', 'security_auditor', 'db_developer')
        THEN '✓ Создана'
        ELSE '✗ Отсутствует'
    END as status
FROM pg_roles
WHERE rolname IN ('app_reader', 'app_writer', 'auditor', 'studio_manager',
'voice_actor', 'security_auditor', 'db_developer');

-- Проверяем RLS
SELECT
    schemaname,
    tablename,
    CASE
        WHEN rowsecurity THEN '✓ Включен'
        ELSE '✗ Выключен'
    END as rls_status
FROM pg_tables
WHERE schemaname = 'app'
  AND tablename IN ('actors', 'contracts', 'recording_sessions', 'audio_archives');

-- Проверяем политики
SELECT
```

```
    schemaname,
    tablename,
    policyname,
    '✓ Настроена' as status
FROM pg_policies
WHERE schemaname = 'app';

-- Инструкция для ручного тестирования
DO $$
BEGIN
    RAISE NOTICE ' ';
    RAISE NOTICE '=== ИНСТРУКЦИЯ ДЛЯ РУЧНОГО ТЕСТИРОВАНИЯ
===';
    RAISE NOTICE 'Откройте НОВОЕ окно Query Tool для тестирования:';
    RAISE NOTICE ' ';
    RAISE NOTICE 'ТЕСТ voice_actor:';
    RAISE NOTICE '1. SET ROLE voice_actor;';
    RAISE NOTICE '2. SET app.current_actor_id = "1";';
    RAISE NOTICE '3. SELECT * FROM app.contracts; -- Только контракты
актёра 1 (без salary_amount)';
    RAISE NOTICE '4. SELECT * FROM app.get_contracts_with_masked_salary();
-- Безопасный просмотр зарплат';
    RAISE NOTICE '5. SELECT public.log_session_start();';
    RAISE NOTICE '6. RESET ROLE;';
    RAISE NOTICE ' ';
    RAISE NOTICE 'ТЕСТ studio_manager:';
    RAISE NOTICE '1. SET ROLE studio_manager;';
    RAISE NOTICE '2. SELECT * FROM app.projects; -- Должен работать';
    RAISE NOTICE '3. SELECT * FROM audit.login_log; -- Должна БЫТЬ
ОШИБКА';
```

```
    RAISE NOTICE '4. RESET ROLE;';
    RAISE NOTICE ' ';
    RAISE NOTICE 'TECT security_auditor:';
    RAISE NOTICE '1. SET ROLE security_auditor;';
    RAISE NOTICE '2. SELECT * FROM audit.login_log; -- Должен работать';
    RAISE NOTICE '3. SELECT * FROM app.contracts; -- Должна БЫТЬ
ОШИБКА';
    RAISE NOTICE '4. RESET ROLE;';
    RAISE NOTICE ' ';
END $$;
```

Приложение Б

```
-- Реализация механизма временных привилегий для роли app_writer
-- Позволяет временно (на N минут) получать право на выполнение
-- чувствительных операций (например, удаление записи)

SET search_path TO app, ref, audit, stg, public;
SET ROLE app_owner;

DO $$
BEGIN
   RAISE NOTICE 'РГЗ: Начало создания системы временных привилегий (Just-In-Time Access)';
END $$;

--
=============================================================
-- 1. Создание таблицы audit.temp_access_log
--
=============================================================

CREATE TABLE IF NOT EXISTS audit.temp_access_log (
   request_id BIGSERIAL PRIMARY KEY,
   request_time TIMESTAMPTZ DEFAULT NOW() NOT NULL,
   caller_role VARCHAR(100) NOT NULL,
   caller_username VARCHAR(100) NOT NULL,
   operation VARCHAR(100) NOT NULL,
   expires_at TIMESTAMPTZ NOT NULL,
```

```
    duration_minutes INTEGER NOT NULL,
    is_active BOOLEAN DEFAULT true,
    used_at TIMESTAMPTZ,
    client_ip INET,
    created_at TIMESTAMPTZ DEFAULT NOW() NOT NULL
);

-- Индексы для быстрого поиска
CREATE INDEX IF NOT EXISTS idx_temp_access_log_caller ON
audit.temp_access_log(caller_username, operation);
CREATE INDEX IF NOT EXISTS idx_temp_access_log_expires ON
audit.temp_access_log(expires_at) WHERE is_active = true;
CREATE INDEX IF NOT EXISTS idx_temp_access_log_active ON
audit.temp_access_log(is_active, expires_at);

-- Комментарии
COMMENT ON TABLE audit.temp_access_log IS
'Журнал запросов временных привилегий (Just-In-Time Access). Записывает
кто, какую операцию и на сколько времени запросил.';

COMMENT ON COLUMN audit.temp_access_log.request_id IS 'Уникальный
идентификатор запроса';
COMMENT ON COLUMN audit.temp_access_log.request_time IS 'Время запроса
временной привилегии';
COMMENT ON COLUMN audit.temp_access_log.caller_role IS 'Роль,
запросившая привилегию';
COMMENT ON COLUMN audit.temp_access_log.caller_username IS
'Пользователь, запросивший привилегию';
COMMENT ON COLUMN audit.temp_access_log.operation IS 'Название
операции (например, delete_contract, delete_actor)';
```

```
COMMENT ON COLUMN audit.temp_access_log.expires_at IS 'Время истечения привилегии';
COMMENT ON COLUMN audit.temp_access_log.duration_minutes IS 'Длительность привилегии в минутах';
COMMENT ON COLUMN audit.temp_access_log.is_active IS 'Активна ли привилегия (автоматически становится false после истечения)';
COMMENT ON COLUMN audit.temp_access_log.used_at IS 'Время последнего использования привилегии';

-- Запрещаем прямой доступ к таблице (только через функции)
REVOKE ALL ON audit.temp_access_log FROM PUBLIC;
REVOKE INSERT, UPDATE, DELETE ON audit.temp_access_log FROM PUBLIC, app_writer, studio_manager;

-- Разрешаем чтение auditor, security_admin и app_writer (только свои записи)
-- Также даём USAGE на схему audit для app_writer и studio_manager
GRANT USAGE ON SCHEMA audit TO app_writer, studio_manager;
GRANT SELECT ON audit.temp_access_log TO auditor, security_admin, app_writer, studio_manager;

-- Создаём роль для логирования временных привилегий
-- Используем существующую роль function_logger или создаём новую только если есть права
DO $$
BEGIN
   IF NOT EXISTS (SELECT 1 FROM pg_roles WHERE rolname = 'temp_privilege_logger') THEN
      BEGIN
         CREATE ROLE temp_privilege_logger WITH NOLOGIN;
      EXCEPTION WHEN insufficient_privilege THEN
```

```
        -- Если нет прав на создание роли, используем существующую роль function_logger
        IF EXISTS (SELECT 1 FROM pg_roles WHERE rolname = 'function_logger') THEN
            RAISE NOTICE 'Используем существующую роль function_logger вместо temp_privilege_logger';
        ELSE
            RAISE EXCEPTION 'Не удалось создать роль temp_privilege_logger и не найдена роль function_logger';
        END IF;
    END;
  END IF;
END $$;

-- Выдаём права на схему и таблицу
DO $$
BEGIN
  IF EXISTS (SELECT 1 FROM pg_roles WHERE rolname = 'temp_privilege_logger') THEN
    GRANT USAGE ON SCHEMA audit TO temp_privilege_logger;
    GRANT INSERT, UPDATE ON audit.temp_access_log TO temp_privilege_logger;
    GRANT USAGE ON SEQUENCE audit.temp_access_log_request_id_seq TO temp_privilege_logger;
  ELSIF EXISTS (SELECT 1 FROM pg_roles WHERE rolname = 'function_logger') THEN
    -- Используем существующую роль function_logger
    GRANT INSERT, UPDATE ON audit.temp_access_log TO function_logger;
    GRANT USAGE ON SEQUENCE audit.temp_access_log_request_id_seq TO function_logger;
```

```
    END IF;
END $$;

--
============================================================
===
-- 2. Справочник разрешённых операций для ролей
--
============================================================
===

CREATE TABLE IF NOT EXISTS audit.allowed_temp_operations (
    operation_id SERIAL PRIMARY KEY,
    operation_name VARCHAR(100) NOT NULL UNIQUE,
    role_name VARCHAR(100) NOT NULL,
    description TEXT,
    is_active BOOLEAN DEFAULT true,
    created_at TIMESTAMPTZ DEFAULT NOW()
);

COMMENT ON TABLE audit.allowed_temp_operations IS
'Справочник разрешённых операций для временных привилегий по ролям';

-- Заполняем справочник разрешённых операций
INSERT INTO audit.allowed_temp_operations (operation_name, role_name,
description) VALUES
    ('delete_contract', 'app_writer', 'Удаление контракта'),
    ('delete_actor', 'app_writer', 'Удаление актёра'),
    ('delete_project', 'app_writer', 'Удаление проекта'),
    ('delete_recording_session', 'app_writer', 'Удаление сессии записи'),
```

```
    ('delete_audio_archive', 'app_writer', 'Удаление аудиоархива')
ON CONFLICT (operation_name) DO UPDATE
    SET description = EXCLUDED.description,
        is_active = TRUE;

-- Запрещаем прямой доступ к справочнику
REVOKE ALL ON audit.allowed_temp_operations FROM PUBLIC;
GRANT SELECT ON audit.allowed_temp_operations TO app_writer,
studio_manager, auditor, security_admin;

--
============================================================
===
-- 3. Функция проверки активной временной привилегии
--
============================================================
===

CREATE OR REPLACE FUNCTION audit.check_temp_privilege(
    p_operation VARCHAR(100)
)
RETURNS BOOLEAN
LANGUAGE plpgsql
VOLATILE
SECURITY DEFINER
SET search_path = audit, public
AS $$
DECLARE
    v_expires_at TIMESTAMPTZ;
    v_is_active BOOLEAN;
```

```
    v_caller_username VARCHAR(100);
    v_request_id BIGINT;
BEGIN
    v_caller_username := session_user;

    -- Проверяем наличие активной привилегии
    SELECT request_id, expires_at, is_active
    INTO v_request_id, v_expires_at, v_is_active
    FROM audit.temp_access_log
    WHERE caller_username = v_caller_username
      AND operation = p_operation
      AND is_active = true
    ORDER BY request_time DESC
    LIMIT 1;

    -- Если привилегия не найдена
    IF NOT FOUND THEN
        RETURN false;
    END IF;

    -- Если привилегия истекла, помечаем её как неактивную
    IF v_expires_at < NOW() THEN
        UPDATE audit.temp_access_log
        SET is_active = false
        WHERE request_id = v_request_id;

        RETURN false;
    END IF;

    -- Обновляем время последнего использования
```

```
    UPDATE audit.temp_access_log
    SET used_at = NOW()
    WHERE request_id = v_request_id;

    RETURN true;
END;
$$;

COMMENT ON FUNCTION audit.check_temp_privilege(VARCHAR) IS
'Проверяет, активна ли временная привилегия для текущего пользователя на
указанную операцию';

--
=============================================================
===
-- 4. Функция запроса временной привилегии
--
=============================================================
===

CREATE OR REPLACE FUNCTION app.request_temp_privilege(
    p_operation_name TEXT,
    p_duration_min INTEGER
)
RETURNS JSONB
LANGUAGE plpgsql
SECURITY DEFINER
SET search_path = app, ref, audit, public
AS $$
DECLARE
```

```
    v_caller_role VARCHAR(100);
    v_caller_username VARCHAR(100);
    v_expires_at TIMESTAMPTZ;
    v_request_id BIGINT;
    v_is_allowed BOOLEAN;
    v_result JSONB;
BEGIN
    -- Получаем информацию о вызывающем
    v_caller_role := COALESCE(current_setting('role', true), session_user);
    v_caller_username := session_user;

    -- ВАЛИДАЦИЯ 1: Проверяем, разрешена ли операция для роли
    SELECT EXISTS (
        SELECT 1
        FROM audit.allowed_temp_operations
        WHERE operation_name = p_operation_name
          AND role_name = v_caller_role
          AND is_active = true
    ) INTO v_is_allowed;

    IF NOT v_is_allowed THEN
        PERFORM audit.log_function_call(
            'app', 'request_temp_privilege',
            jsonb_build_object(
                'operation_name', p_operation_name,
                'duration_min', p_duration_min,
                'caller_role', v_caller_role
            ),
            NULL,
            false,
```

```
        'ОШИБКА: Операция ' || p_operation_name || ' не разрешена для роли ' ||
v_caller_role,
        NULL
      );
      RAISE EXCEPTION 'Операция % не разрешена для роли %',
p_operation_name, v_caller_role;
   END IF;

   -- ВАЛИДАЦИЯ 2: Проверяем длительность (от 1 до 60 минут)
   IF p_duration_min IS NULL OR p_duration_min < 1 OR p_duration_min > 60
THEN
      PERFORM audit.log_function_call(
         'app', 'request_temp_privilege',
         jsonb_build_object(
            'operation_name', p_operation_name,
            'duration_min', p_duration_min
         ),
         NULL,
         false,
         'ОШИБКА: Длительность должна быть от 1 до 60 минут',
         NULL
      );
      RAISE EXCEPTION 'Длительность должна быть от 1 до 60 минут,
получено: %', p_duration_min;
   END IF;

   -- Вычисляем время истечения
   v_expires_at := NOW() + (p_duration_min || ' minutes')::INTERVAL;

   -- Деактивируем предыдущие активные привилегии для этой операции
```

```
UPDATE audit.temp_access_log
SET is_active = false
WHERE caller_username = v_caller_username
  AND operation = p_operation_name
  AND is_active = true;

-- Записываем запрос в audit.temp_access_log
INSERT INTO audit.temp_access_log (
    caller_role,
    caller_username,
    operation,
    expires_at,
    duration_minutes,
    is_active,
    client_ip
) VALUES (
    v_caller_role,
    v_caller_username,
    p_operation_name,
    v_expires_at,
    p_duration_min,
    true,
    inet_client_addr()
)
RETURNING request_id INTO v_request_id;

-- Устанавливаем временную метку истечения через GUC
PERFORM set_config('app.temp_privilege_until', v_expires_at::TEXT, false);

-- Формируем результат
```

```
    v_result := jsonb_build_object(
        'request_id', v_request_id,
        'operation', p_operation_name,
        'expires_at', v_expires_at,
        'duration_minutes', p_duration_min,
        'status', 'granted'
    );

    -- Логируем успешный вызов
    PERFORM audit.log_function_call(
        'app', 'request_temp_privilege',
        jsonb_build_object(
            'operation_name', p_operation_name,
            'duration_min', p_duration_min
        ),
        v_result,
        true,
        NULL,
        NULL
    );

    RETURN v_result;

EXCEPTION
    WHEN OTHERS THEN
        -- Логируем ошибку
        PERFORM audit.log_function_call(
            'app', 'request_temp_privilege',
            jsonb_build_object(
                'operation_name', p_operation_name,
```

```
            'duration_min', p_duration_min
        ),
        NULL,
        false,
        'ОШИБКА: ' || SQLERRM,
        NULL
    );

    RAISE;
END;
$$;

COMMENT ON FUNCTION app.request_temp_privilege(TEXT, INTEGER) IS
'SECURITY DEFINER: Запрашивает временную привилегию на выполнение
чувствительной операции. Проверяет права роли, записывает в
audit.temp_access_log и устанавливает GUC app.temp_privilege_until.';

-- Даём доступ к функции
GRANT EXECUTE ON FUNCTION app.request_temp_privilege(TEXT,
INTEGER) TO app_writer, studio_manager;
GRANT EXECUTE ON FUNCTION audit.check_temp_privilege(VARCHAR) TO
app_writer, studio_manager, app_owner;

-- Назначаем роль для логирования (если она существует)
DO $$
BEGIN
   IF EXISTS (SELECT 1 FROM pg_roles WHERE rolname =
'temp_privilege_logger') THEN
    GRANT temp_privilege_logger TO app_owner;
```

```
    ELSIF EXISTS (SELECT 1 FROM pg_roles WHERE rolname =
'function_logger') THEN
        -- function_logger уже назначена app_owner в ЛР2
        NULL;
    END IF;
END $$;

DO $$
BEGIN
    RAISE NOTICE 'РГЗ: Таблица audit.temp_access_log и функция
request_temp_privilege успешно созданы';
END $$;

RESET ROLE;



--
==========================================================
===
-- SECURITY DEFINER-функции для безопасного удаления
-- с проверкой временных привилегий
--
==========================================================
===

SET search_path TO app, ref, audit, stg, public;
SET ROLE app_owner;

DO $$
BEGIN
```

```
    RAISE NOTICE 'РГЗ: Создание функций безопасного удаления с проверкой временных привилегий';
END $$;

--
=============================================================
===
-- ФУНКЦИЯ 1: safe_delete_contract
-- Безопасное удаление контракта с проверкой временной привилегии
--
=============================================================
===

CREATE OR REPLACE FUNCTION app.safe_delete_contract(
    p_contract_id INTEGER
)
RETURNS BOOLEAN
LANGUAGE plpgsql
SECURITY DEFINER
SET search_path = app, ref, audit, public
AS $$
DECLARE
    v_start_time TIMESTAMPTZ;
    v_end_time TIMESTAMPTZ;
    v_execution_time NUMERIC;
    v_has_privilege BOOLEAN;
    v_contract_exists BOOLEAN;
    v_input_params JSONB;
    v_return_value JSONB;
BEGIN
```

```
    v_start_time := clock_timestamp();

    -- Формируем JSON с параметрами
    v_input_params := jsonb_build_object(
        'contract_id', p_contract_id,
        'operation', 'delete_contract'
    );

    -- ПРОВЕРКА 1: Проверяем наличие временной привилегии
    SELECT audit.check_temp_privilege('delete_contract') INTO v_has_privilege;

    IF NOT v_has_privilege THEN
        PERFORM audit.log_function_call(
            'app', 'safe_delete_contract',
            v_input_params,
            NULL,
            false,
            'ОШИБКА: Временная привилегия на удаление контракта не активна или истекла. Вызовите app.request_temp_privilege("delete_contract", duration_min)',
            NULL
        );
        RAISE EXCEPTION 'Временная привилегия на удаление контракта не активна или истекла. Вызовите app.request_temp_privilege("delete_contract", duration_min)';
    END IF;

    -- ПРОВЕРКА 2: Проверяем существование контракта
    SELECT EXISTS (
        SELECT 1 FROM app.contracts WHERE contract_id = p_contract_id
```

```
    ) INTO v_contract_exists;

    IF NOT v_contract_exists THEN
        PERFORM audit.log_function_call(
            'app', 'safe_delete_contract',
            v_input_params,
            NULL,
            false,
            'ОШИБКА: Контракт с ID ' || p_contract_id || ' не найден',
            NULL
        );
        RAISE EXCEPTION 'Контракт с ID % не найден', p_contract_id;
    END IF;

    -- ПРОВЕРКА 3: Проверяем, нет ли связанных записей (сессии записи)
    IF EXISTS (
        SELECT 1 FROM app.recording_sessions WHERE contract_id = p_contract_id
    ) THEN
        PERFORM audit.log_function_call(
            'app', 'safe_delete_contract',
            v_input_params,
            NULL,
            false,
            'ОШИБКА: Невозможно удалить контракт, так как существуют
связанные сессии записи',
            NULL
        );
        RAISE EXCEPTION 'Невозможно удалить контракт с ID %, так как
существуют связанные сессии записи', p_contract_id;
    END IF;
```

```
    -- УДАЛЕНИЕ КОНТРАКТА
    DELETE FROM app.contracts
    WHERE contract_id = p_contract_id;

    -- Формируем возвращаемое значение
    v_return_value := jsonb_build_object(
        'contract_id', p_contract_id,
        'deleted', true,
        'message', 'Контракт успешно удалён'
    );

    -- Вычисляем время выполнения
    v_end_time := clock_timestamp();
    v_execution_time := EXTRACT(EPOCH FROM (v_end_time - v_start_time)) *
1000;

    -- ЛОГИРОВАНИЕ УСПЕШНОГО УДАЛЕНИЯ
    PERFORM audit.log_function_call(
        'app', 'safe_delete_contract',
        v_input_params,
        v_return_value,
        true,
        NULL,
        v_execution_time
    );

    RETURN true;

EXCEPTION
```

```
    WHEN OTHERS THEN
        v_end_time := clock_timestamp();
        v_execution_time := EXTRACT(EPOCH FROM (v_end_time - v_start_time))
* 1000;

        PERFORM audit.log_function_call(
            'app', 'safe_delete_contract',
            v_input_params,
            NULL,
            false,
            'ОШИБКА: ' || SQLERRM,
            v_execution_time
        );

        RAISE;
END;
$$;

COMMENT ON FUNCTION app.safe_delete_contract(INTEGER) IS
'SECURITY DEFINER: Безопасное удаление контракта с проверкой временной
привилегии. Требует активную временную привилегию delete_contract.';

--
==========================================================
===
-- ФУНКЦИЯ 2: safe_delete_actor
-- Безопасное удаление актёра с проверкой временной привилегии
--
==========================================================
===
```

```
CREATE OR REPLACE FUNCTION app.safe_delete_actor(
    p_actor_id INTEGER
)
RETURNS BOOLEAN
LANGUAGE plpgsql
SECURITY DEFINER
SET search_path = app, ref, audit, public
AS $$
DECLARE
    v_start_time TIMESTAMPTZ;
    v_end_time TIMESTAMPTZ;
    v_execution_time NUMERIC;
    v_has_privilege BOOLEAN;
    v_actor_exists BOOLEAN;
    v_input_params JSONB;
    v_return_value JSONB;
BEGIN
    v_start_time := clock_timestamp();

    -- Формируем JSON с параметрами
    v_input_params := jsonb_build_object(
        'actor_id', p_actor_id,
        'operation', 'delete_actor'
    );

    -- ПРОВЕРКА 1: Проверяем наличие временной привилегии
    SELECT audit.check_temp_privilege('delete_actor') INTO v_has_privilege;

    IF NOT v_has_privilege THEN
```

```
        PERFORM audit.log_function_call(
            'app', 'safe_delete_actor',
            v_input_params,
            NULL,
            false,
            'ОШИБКА: Временная привилегия на удаление актёра не активна или истекла. Вызовите app.request_temp_privilege("delete_actor", duration_min)',
            NULL
        );
        RAISE EXCEPTION 'Временная привилегия на удаление актёра не активна или истекла. Вызовите app.request_temp_privilege("delete_actor", duration_min)';
    END IF;

    -- ПРОВЕРКА 2: Проверяем существование актёра
    SELECT EXISTS (
        SELECT 1 FROM app.actors WHERE actor_id = p_actor_id
    ) INTO v_actor_exists;

    IF NOT v_actor_exists THEN
        PERFORM audit.log_function_call(
            'app', 'safe_delete_actor',
            v_input_params,
            NULL,
            false,
            'ОШИБКА: Актёр с ID ' || p_actor_id || ' не найден',
            NULL
        );
        RAISE EXCEPTION 'Актёр с ID % не найден', p_actor_id;
    END IF;
```

```
    -- ПРОВЕРКА 3: Проверяем, нет ли связанных контрактов
    IF EXISTS (
        SELECT 1 FROM app.contracts WHERE actor_id = p_actor_id
    ) THEN
        PERFORM audit.log_function_call(
            'app', 'safe_delete_actor',
            v_input_params,
            NULL,
            false,
            'ОШИБКА: Невозможно удалить актёра, так как существуют связанные контракты',
            NULL
        );
        RAISE EXCEPTION 'Невозможно удалить актёра с ID %, так как существуют связанные контракты', p_actor_id;
    END IF;

    -- УДАЛЕНИЕ АКТЁРА
    DELETE FROM app.actors
    WHERE actor_id = p_actor_id;

    -- Формируем возвращаемое значение
    v_return_value := jsonb_build_object(
        'actor_id', p_actor_id,
        'deleted', true,
        'message', 'Актёр успешно удалён'
    );

    -- Вычисляем время выполнения
    v_end_time := clock_timestamp();
```

```
    v_execution_time := EXTRACT(EPOCH FROM (v_end_time - v_start_time)) *
1000;

    -- ЛОГИРОВАНИЕ УСПЕШНОГО УДАЛЕНИЯ
    PERFORM audit.log_function_call(
        'app', 'safe_delete_actor',
        v_input_params,
        v_return_value,
        true,
        NULL,
        v_execution_time
    );

    RETURN true;

EXCEPTION
    WHEN OTHERS THEN
        v_end_time := clock_timestamp();
        v_execution_time := EXTRACT(EPOCH FROM (v_end_time - v_start_time))
* 1000;

        PERFORM audit.log_function_call(
            'app', 'safe_delete_actor',
            v_input_params,
            NULL,
            false,
            'ОШИБКА: ' || SQLERRM,
            v_execution_time
        );
```

```
        RAISE;
END;
$$;

COMMENT ON FUNCTION app.safe_delete_actor(INTEGER) IS
'SECURITY DEFINER: Безопасное удаление актёра с проверкой временной привилегии. Требует активную временную привилегию delete_actor.';

--
================================================================
-- ФУНКЦИЯ 3: safe_delete_recording_session
-- Безопасное удаление сессии записи с проверкой временной привилегии
--
================================================================

CREATE OR REPLACE FUNCTION app.safe_delete_recording_session(
    p_session_id INTEGER
)
RETURNS BOOLEAN
LANGUAGE plpgsql
SECURITY DEFINER
SET search_path = app, ref, audit, public
AS $$
DECLARE
    v_start_time TIMESTAMPTZ;
    v_end_time TIMESTAMPTZ;
    v_execution_time NUMERIC;
    v_has_privilege BOOLEAN;
```

```
    v_session_exists BOOLEAN;
    v_input_params JSONB;
    v_return_value JSONB;
BEGIN
    v_start_time := clock_timestamp();

    -- Формируем JSON с параметрами
    v_input_params := jsonb_build_object(
        'session_id', p_session_id,
        'operation', 'delete_recording_session'
    );

    -- ПРОВЕРКА 1: Проверяем наличие временной привилегии
    SELECT audit.check_temp_privilege('delete_recording_session') INTO
v_has_privilege;

    IF NOT v_has_privilege THEN
        PERFORM audit.log_function_call(
            'app', 'safe_delete_recording_session',
            v_input_params,
            NULL,
            false,
            'ОШИБКА: Временная привилегия на удаление сессии записи не
активна или истекла. Вызовите
app.request_temp_privilege("delete_recording_session", duration_min)',
            NULL
        );
        RAISE EXCEPTION 'Временная привилегия на удаление сессии записи не
активна или истекла. Вызовите
app.request_temp_privilege("delete_recording_session", duration_min)';
```

```
    END IF;

    -- ПРОВЕРКА 2: Проверяем существование сессии
    SELECT EXISTS (
        SELECT 1 FROM app.recording_sessions WHERE session_id = p_session_id
    ) INTO v_session_exists;

    IF NOT v_session_exists THEN
        PERFORM audit.log_function_call(
            'app', 'safe_delete_recording_session',
            v_input_params,
            NULL,
            false,
            'ОШИБКА: Сессия записи с ID ' || p_session_id || ' не найдена',
            NULL
        );
        RAISE EXCEPTION 'Сессия записи с ID % не найдена', p_session_id;
    END IF;

    -- ПРОВЕРКА 3: Проверяем, нет ли связанных аудиоархивов
    IF EXISTS (
        SELECT 1 FROM app.audio_archives WHERE session_id = p_session_id
    ) THEN
        PERFORM audit.log_function_call(
            'app', 'safe_delete_recording_session',
            v_input_params,
            NULL,
            false,
            'ОШИБКА: Невозможно удалить сессию записи, так как существуют связанные аудиоархивы',
```

```
            NULL
        );
        RAISE EXCEPTION 'Невозможно удалить сессию записи с ID %, так как существуют связанные аудиоархивы', p_session_id;
    END IF;

    -- УДАЛЕНИЕ СЕССИИ
    DELETE FROM app.recording_sessions
    WHERE session_id = p_session_id;

    -- Формируем возвращаемое значение
    v_return_value := jsonb_build_object(
        'session_id', p_session_id,
        'deleted', true,
        'message', 'Сессия записи успешно удалена'
    );

    -- Вычисляем время выполнения
    v_end_time := clock_timestamp();
    v_execution_time := EXTRACT(EPOCH FROM (v_end_time - v_start_time)) * 1000;

    -- ЛОГИРОВАНИЕ УСПЕШНОГО УДАЛЕНИЯ
    PERFORM audit.log_function_call(
        'app', 'safe_delete_recording_session',
        v_input_params,
        v_return_value,
        true,
        NULL,
        v_execution_time
```

```
    );

    RETURN true;

EXCEPTION
    WHEN OTHERS THEN
        v_end_time := clock_timestamp();
        v_execution_time := EXTRACT(EPOCH FROM (v_end_time - v_start_time))
* 1000;

        PERFORM audit.log_function_call(
            'app', 'safe_delete_recording_session',
            v_input_params,
            NULL,
            false,
            'ОШИБКА: ' || SQLERRM,
            v_execution_time
        );

        RAISE;
END;
$$;

COMMENT ON FUNCTION app.safe_delete_recording_session(INTEGER) IS
'SECURITY DEFINER: Безопасное удаление сессии записи с проверкой
временной привилегии. Требует активную временную привилегию
delete_recording_session.';

-- Даём доступ к функциям
```

```
GRANT EXECUTE ON FUNCTION app.safe_delete_contract(INTEGER) TO
app_writer, studio_manager;
GRANT EXECUTE ON FUNCTION app.safe_delete_actor(INTEGER) TO
app_writer, studio_manager;
GRANT EXECUTE ON FUNCTION
app.safe_delete_recording_session(INTEGER) TO app_writer, studio_manager;

DO $$
BEGIN
   RAISE NOTICE 'РГЗ: Функции безопасного удаления успешно созданы:';
   RAISE NOTICE '   - app.safe_delete_contract';
   RAISE NOTICE '   - app.safe_delete_actor';
   RAISE NOTICE '   - app.safe_delete_recording_session';
END $$;

RESET ROLE;



--
==========================================================
===
-- Тесты системы временных привилегий (Just-In-Time Access)
--
==========================================================
===
-- Тестирование: запрос → выполнение → попытка после истечения →
ошибка

SET search_path TO app, ref, audit, stg, public;
```

```
DO $$
BEGIN
   RAISE NOTICE '========================================';
   RAISE NOTICE 'РГЗ: Начало тестирования системы временных привилегий';
   RAISE NOTICE '========================================';
END $$;

--
================================================
===
-- ТЕСТ 1: Запрос временной привилегии
--
================================================
===

DO $$
BEGIN
   RAISE NOTICE '';
   RAISE NOTICE '=== ТЕСТ 1: Запрос временной привилегии ===';
END $$;

-- Подключаемся от имени app_writer
SET ROLE app_writer;

-- Сохраняем имя пользователя для использования в тестах
DO $$
BEGIN
   PERFORM set_config('test.caller_username', session_user, false);
END $$;
```

```
-- Устанавливаем контекст сегмента (необходимо для RLS)
-- Пропускаем установку контекста, если функция не доступна (тесты могут работать и без неё)
DO $$
BEGIN
  -- Пытаемся установить контекст, но не критично если не получится
  BEGIN
    -- Проверяем, существует ли функция
    IF EXISTS (
      SELECT 1 FROM pg_proc p
      JOIN pg_namespace n ON p.pronamespace = n.oid
      WHERE n.nspname = 'app' AND p.proname = 'set_session_ctx'
    ) THEN
      -- Вызываем функцию с правильной сигнатурой
      EXECUTE 'SELECT app.set_session_ctx($1)' USING 1;
      RAISE NOTICE 'Контекст сегмента установлен';
    ELSE
      RAISE NOTICE 'Функция app.set_session_ctx не найдена (продолжаем без контекста)';
    END IF;
  EXCEPTION
    WHEN OTHERS THEN
      RAISE NOTICE 'Ошибка установки контекста: % (продолжаем без контекста)', SQLERRM;
  END;
END $$;

-- Тест 1.1: Запрос привилегии на удаление контракта на 5 минут
DO $$
DECLARE
```

```
    v_result JSONB;
BEGIN
    v_result := app.request_temp_privilege('delete_contract', 5);
    RAISE NOTICE '✅ ТЕСТ 1.1 ПРОЙДЕН: Привилегия запрошена успешно';
    RAISE NOTICE '   Request ID: %', v_result->>'request_id';
    RAISE NOTICE '   Операция: %', v_result->>'operation';
    RAISE NOTICE '   Истекает: %', v_result->>'expires_at';
    RAISE NOTICE '   Длительность: % минут', v_result->>'duration_minutes';
EXCEPTION
    WHEN OTHERS THEN
        RAISE NOTICE '❌ ТЕСТ 1.1 ПРОВАЛЕН: %', SQLERRM;
END $$;

-- Проверяем запись в audit.temp_access_log
DO $$
DECLARE
    v_count INTEGER;
    v_log_record RECORD;
BEGIN
    SELECT COUNT(*) INTO v_count
    FROM audit.temp_access_log
    WHERE caller_username = session_user
      AND operation = 'delete_contract'
      AND is_active = true;

    IF v_count > 0 THEN
        SELECT * INTO v_log_record
        FROM audit.temp_access_log
        WHERE caller_username = session_user
```

```
        AND operation = 'delete_contract'
        AND is_active = true
    ORDER BY request_time DESC
    LIMIT 1;

    RAISE NOTICE '✅ ТЕСТ 1.2 ПРОЙДЕН: Запись в audit.temp_access_log
создана';
    RAISE NOTICE '   Request ID: %', v_log_record.request_id;
    RAISE NOTICE '   Caller: %', v_log_record.caller_username;
    RAISE NOTICE '   Operation: %', v_log_record.operation;
    RAISE NOTICE '   Expires at: %', v_log_record.expires_at;
  ELSE
    RAISE NOTICE '❌ ТЕСТ 1.2 ПРОВАЛЕН: Запись в audit.temp_access_log
не найдена';
  END IF;
END $$;

--
==========================================================
===
-- ТЕСТ 2: Выполнение операции с активной привилегией
--
==========================================================
===

DO $$
BEGIN
  RAISE NOTICE '';
```

```
    RAISE NOTICE '=== ТЕСТ 2: Выполнение операции с активной привилегией
===';
END $$;

-- Тест 2.1: Находим существующий контракт для удаления
DO $$
DECLARE
    v_contract_id INTEGER;
    v_test_project_id INTEGER;
    v_test_actor_id INTEGER;
BEGIN
    -- Сначала пытаемся найти существующий контракт без сессий
(используем RESET ROLE для обхода RLS)
    RESET ROLE;
    SELECT contract_id INTO v_contract_id
    FROM app.contracts c
    WHERE NOT EXISTS (
        SELECT 1 FROM app.recording_sessions rs
        WHERE rs.contract_id = c.contract_id
    )
    LIMIT 1;
    SET ROLE app_writer;

    -- Если не нашли, пытаемся создать новый
    IF v_contract_id IS NULL THEN
        -- Находим существующий проект и актёра (используем RESET ROLE для
обхода RLS)
        RESET ROLE;
        SELECT project_id INTO v_test_project_id FROM app.projects LIMIT 1;
```

```
    SELECT actor_id INTO v_test_actor_id FROM app.actors WHERE is_active =
true LIMIT 1;
    SET ROLE app_writer;

    IF v_test_project_id IS NULL OR v_test_actor_id IS NULL THEN
        RAISE NOTICE '⚠️ Не найдены тестовые данные для создания
контракта';
        RETURN;
    END IF;

    -- Создаём тестовый контракт
    BEGIN
        v_contract_id := app.create_contract(
            p_project_id := v_test_project_id,
            p_actor_id := v_test_actor_id,
            p_contract_number := 'TEST-DELETE-' || extract(epoch from
now())::TEXT,
            p_role_description := 'Тестовый контракт для удаления',
            p_salary_amount := 1000.00,
            p_currency := 'USD'
        );

        RAISE NOTICE '✅ Тестовый контракт создан: ID = %', v_contract_id;
    EXCEPTION
        WHEN OTHERS THEN
            RAISE NOTICE '⚠️ Ошибка создания тестового контракта: %',
SQLERRM;
            RETURN;
    END;
```

```
    ELSE
        RAISE NOTICE '✅ Используем существующий контракт без сессий: ID =
%', v_contract_id;
    END IF;

    -- Сохраняем ID для последующих тестов
    PERFORM set_config('test.contract_id', v_contract_id::TEXT, false);
END $$;

-- Тест 2.2: Удаление контракта с активной привилегией
DO $$
DECLARE
    v_contract_id INTEGER;
    v_result BOOLEAN;
BEGIN
    v_contract_id := current_setting('test.contract_id', true)::INTEGER;

    IF v_contract_id IS NULL THEN
        RAISE NOTICE '❌ ТЕСТ 2.2 ПРОВАЛЕН: Нет тестового контракта';
        RETURN;
    END IF;

    -- Проверяем, что привилегия активна
    IF NOT audit.check_temp_privilege('delete_contract') THEN
        RAISE NOTICE '⚠️ Привилегия не активна, запрашиваем заново';
        PERFORM app.request_temp_privilege('delete_contract', 5);
    END IF;

    -- Удаляем контракт
```

```
    v_result := app.safe_delete_contract(v_contract_id);

    IF v_result THEN
        RAISE NOTICE '✅ ТЕСТ 2.2 ПРОЙДЕН: Контракт успешно удалён с активной привилегией';
    ELSE
        RAISE NOTICE '❌ ТЕСТ 2.2 ПРОВАЛЕН: Удаление не выполнено';
    END IF;
EXCEPTION
    WHEN OTHERS THEN
        RAISE NOTICE '❌ ТЕСТ 2.2 ПРОВАЛЕН: %', SQLERRM;
END $$;

--
================================================================
-- ТЕСТ 3: Попытка выполнения операции без привилегии
--
================================================================

DO $$
BEGIN
    RAISE NOTICE '';
    RAISE NOTICE '=== ТЕСТ 3: Попытка выполнения операции без привилегии ===';
END $$;

-- Тест 3.1: Попытка удаления без запроса привилегии
```

```
DO $$
DECLARE
    v_test_project_id INTEGER;
    v_test_actor_id INTEGER;
    v_contract_id INTEGER;
    v_error_occurred BOOLEAN := false;
BEGIN
    -- Находим существующий контракт или создаём новый (используем RESET ROLE для обхода RLS)
    RESET ROLE;
    SELECT contract_id INTO v_contract_id
    FROM app.contracts c
    WHERE NOT EXISTS (
        SELECT 1 FROM app.recording_sessions rs
        WHERE rs.contract_id = c.contract_id
    )
    LIMIT 1;
    SET ROLE app_writer;

    -- Если не нашли, пытаемся создать новый
    IF v_contract_id IS NULL THEN
        RESET ROLE;
        SELECT project_id INTO v_test_project_id FROM app.projects LIMIT 1;
        SELECT actor_id INTO v_test_actor_id FROM app.actors WHERE is_active = true LIMIT 1;
        SET ROLE app_writer;

        IF v_test_project_id IS NULL OR v_test_actor_id IS NULL THEN
            RAISE NOTICE '⚠ Не найдены тестовые данные для создания контракта';
```

```
            RETURN;
        END IF;

        BEGIN
            v_contract_id := app.create_contract(
                p_project_id := v_test_project_id,
                p_actor_id := v_test_actor_id,
                p_contract_number := 'TEST-NO-PRIV-' || extract(epoch from
now())::TEXT,
                p_role_description := 'Тестовый контракт для теста без привилегии',
                p_salary_amount := 1000.00,
                p_currency := 'USD'
            );
        EXCEPTION
            WHEN OTHERS THEN
                RAISE NOTICE '⚠ Ошибка создания тестового контракта: %',
SQLERRM;
                RETURN;
        END;
    END IF;

    -- Деактивируем все активные привилегии (используем RESET ROLE для
обхода ограничений)
    RESET ROLE;
    UPDATE audit.temp_access_log
    SET is_active = false
    WHERE caller_username = current_setting('test.caller_username', true)
      AND operation = 'delete_contract';
    SET ROLE app_writer;
```

```
    -- Пытаемся удалить без привилегии
    BEGIN
        PERFORM app.safe_delete_contract(v_contract_id);
        v_error_occurred := false;
    EXCEPTION
        WHEN OTHERS THEN
            v_error_occurred := true;
            IF SQLERRM LIKE '%Временная привилегия%' THEN
                RAISE NOTICE '✅ ТЕСТ 3.1 ПРОЙДЕН: Операция корректно
заблокирована без привилегии';
                RAISE NOTICE '   Ошибка: %', SQLERRM;
            ELSE
                RAISE NOTICE '❌ ТЕСТ 3.1 ПРОВАЛЕН: Неожиданная ошибка: %',
SQLERRM;
            END IF;
    END;

    IF NOT v_error_occurred THEN
        RAISE NOTICE '❌ ТЕСТ 3.1 ПРОВАЛЕН: Операция выполнилась без
привилегии (не должно быть!)';
    END IF;
END $$;

--
==========================================================
===
-- ТЕСТ 4: Попытка после истечения привилегии
```

```
--
==========================================================
===

DO $$
BEGIN
    RAISE NOTICE '';
    RAISE NOTICE '=== ТЕСТ 4: Попытка после истечения привилегии ===';
END $$;

-- Тест 4.1: Запрашиваем привилегию на очень короткое время (1 секунда =
0.017 минут)
DO $$
DECLARE
    v_result JSONB;
    v_test_project_id INTEGER;
    v_test_actor_id INTEGER;
    v_contract_id INTEGER;
    v_error_occurred BOOLEAN := false;
BEGIN
    -- Находим существующий контракт или создаём новый (используем RESET
ROLE для обхода RLS)
    RESET ROLE;
    SELECT contract_id INTO v_contract_id
    FROM app.contracts c
    WHERE NOT EXISTS (
        SELECT 1 FROM app.recording_sessions rs
        WHERE rs.contract_id = c.contract_id
    )
    LIMIT 1;
```

```
    SET ROLE app_writer;

    -- Если не нашли, пытаемся создать новый
    IF v_contract_id IS NULL THEN
        RESET ROLE;
        SELECT project_id INTO v_test_project_id FROM app.projects LIMIT 1;
        SELECT actor_id INTO v_test_actor_id FROM app.actors WHERE is_active =
true LIMIT 1;
        SET ROLE app_writer;

        IF v_test_project_id IS NULL OR v_test_actor_id IS NULL THEN
            RAISE NOTICE '⚠ Не найдены тестовые данные для создания
контракта';
            RETURN;
        END IF;

        BEGIN
            v_contract_id := app.create_contract(
                p_project_id := v_test_project_id,
                p_actor_id := v_test_actor_id,
                p_contract_number := 'TEST-EXPIRED-' || extract(epoch from
now())::TEXT,
                p_role_description := 'Тестовый контракт для теста истечения',
                p_salary_amount := 1000.00,
                p_currency := 'USD'
            );
        EXCEPTION
            WHEN OTHERS THEN
```

```
            RAISE NOTICE '⚠️ Ошибка создания тестового контракта: %',
SQLERRM;
            RETURN;
        END;
    END IF;

    -- Запрашиваем привилегию на минимальное время (1 минута)
    v_result := app.request_temp_privilege('delete_contract', 1);
    RAISE NOTICE '✅ Привилегия запрошена на 1 минуту';
    RAISE NOTICE '   Истекает: %', v_result->>'expires_at';

    -- Вручную истекаем привилегию (устанавливаем expires_at в прошлое)
    -- Используем RESET ROLE для обхода ограничений
    RESET ROLE;
    UPDATE audit.temp_access_log
    SET expires_at = NOW() - INTERVAL '1 minute',
        is_active = false
    WHERE request_id = (
        SELECT request_id
        FROM audit.temp_access_log
        WHERE caller_username = current_setting('test.caller_username', true)
          AND operation = 'delete_contract'
          AND is_active = true
        ORDER BY request_time DESC
        LIMIT 1
    );
    SET ROLE app_writer;

    RAISE NOTICE 'Привилегия принудительно истекла';
```

```
    -- Небольшая задержка не требуется, так как мы уже установили
expires_at в прошлое

    -- Пытаемся удалить после истечения
    BEGIN
        PERFORM app.safe_delete_contract(v_contract_id);
        v_error_occurred := false;
    EXCEPTION
        WHEN OTHERS THEN
            v_error_occurred := true;
            IF SQLERRM LIKE '%Временная привилегия%' OR SQLERRM LIKE
'%истекла%' THEN
                RAISE NOTICE '✅ ТЕСТ 4.1 ПРОЙДЕН: Операция корректно
заблокирована после истечения';
                RAISE NOTICE '   Ошибка: %', SQLERRM;
            ELSE
                RAISE NOTICE '❌ ТЕСТ 4.1 ПРОВАЛЕН: Неожиданная ошибка: %',
SQLERRM;
            END IF;
    END;

    IF NOT v_error_occurred THEN
        RAISE NOTICE '❌ ТЕСТ 4.1 ПРОВАЛЕН: Операция выполнилась после
истечения (не должно быть!)';
    END IF;
END $$;
```

```
--
==========================================================
===
-- ТЕСТ 5: Проверка логирования в audit.temp_access_log
--
==========================================================
===

DO $$
BEGIN
   RAISE NOTICE '';
   RAISE NOTICE '=== ТЕСТ 5: Проверка логирования ===';
END $$;

-- Тест 5.1: Проверяем все записи в audit.temp_access_log
DO $$
DECLARE
   v_count INTEGER;
   v_active_count INTEGER;
   v_expired_count INTEGER;
BEGIN
   SELECT COUNT(*) INTO v_count
   FROM audit.temp_access_log
   WHERE caller_username = session_user;

   SELECT COUNT(*) INTO v_active_count
   FROM audit.temp_access_log
   WHERE caller_username = session_user
     AND is_active = true
     AND expires_at >= NOW();
```

```
    SELECT COUNT(*) INTO v_expired_count
    FROM audit.temp_access_log
    WHERE caller_username = session_user
      AND (is_active = false OR expires_at < NOW());

    RAISE NOTICE '✅ ТЕСТ 5.1 ПРОЙДЕН: Логирование работает';
    RAISE NOTICE '   Всего записей: %', v_count;
    RAISE NOTICE '   Активных: %', v_active_count;
    RAISE NOTICE '   Истекших: %', v_expired_count;

    IF v_count = 0 THEN
        RAISE NOTICE '⚠️ Внимание: Нет записей в audit.temp_access_log';
    END IF;
END $$;

-- Тест 5.2: Просмотр последних записей
DO $$
DECLARE
    v_record RECORD;
BEGIN
    RAISE NOTICE '';
    RAISE NOTICE 'Последние 5 записей в audit.temp_access_log:';

    FOR v_record IN
        SELECT
            request_id,
            request_time,
            caller_username,
```

```
            operation,
            expires_at,
            duration_minutes,
            is_active,
            used_at
        FROM audit.temp_access_log
        WHERE caller_username = session_user
        ORDER BY request_time DESC
        LIMIT 5
    LOOP
        RAISE NOTICE '  [%] %: % (expires: %, active: %, used: %)',
            v_record.request_id,
            v_record.operation,
            v_record.request_time,
            v_record.expires_at,
            v_record.is_active,
            COALESCE(v_record.used_at::TEXT, 'never');
    END LOOP;
END $$;

--
=============================================================
===
-- ТЕСТ 6: Проверка валидации параметров
--
=============================================================
===

DO $$
BEGIN
```

```
    RAISE NOTICE '';
    RAISE NOTICE '=== ТЕСТ 6: Проверка валидации параметров ===';
END $$;

-- Тест 6.1: Попытка запросить неразрешённую операцию
DO $$
DECLARE
    v_error_occurred BOOLEAN := false;
BEGIN
    BEGIN
        PERFORM app.request_temp_privilege('delete_everything', 5);
        v_error_occurred := false;
    EXCEPTION
        WHEN OTHERS THEN
            v_error_occurred := true;
            IF SQLERRM LIKE '%не разрешена%' THEN
                RAISE NOTICE '✅ ТЕСТ 6.1 ПРОЙДЕН: Неразрешённая операция корректно отклонена';
                RAISE NOTICE '   Ошибка: %', SQLERRM;
            ELSE
                RAISE NOTICE '❌ ТЕСТ 6.1 ПРОВАЛЕН: Неожиданная ошибка: %', SQLERRM;
            END IF;
    END;

    IF NOT v_error_occurred THEN
        RAISE NOTICE '❌ ТЕСТ 6.1 ПРОВАЛЕН: Неразрешённая операция была принята (не должно быть!)';
    END IF;
```

```
END $$;

-- Тест 6.2: Попытка запросить привилегию с недопустимой длительностью
DO $$
DECLARE
    v_error_occurred BOOLEAN := false;
BEGIN
    BEGIN
        PERFORM app.request_temp_privilege('delete_contract', 120); -- > 60 минут
        v_error_occurred := false;
    EXCEPTION
        WHEN OTHERS THEN
            v_error_occurred := true;
            IF SQLERRM LIKE '%Длительность%' THEN
                RAISE NOTICE '✅ ТЕСТ 6.2 ПРОЙДЕН: Недопустимая длительность
корректно отклонена';
                RAISE NOTICE '   Ошибка: %', SQLERRM;
            ELSE
                RAISE NOTICE '❌ ТЕСТ 6.2 ПРОВАЛЕН: Неожиданная ошибка: %',
SQLERRM;
            END IF;
    END;

    IF NOT v_error_occurred THEN
        RAISE NOTICE '❌ ТЕСТ 6.2 ПРОВАЛЕН: Недопустимая длительность
была принята (не должно быть!)';
    END IF;
END $$;
```

```
-- Тест 6.3: Попытка запросить привилегию с нулевой длительностью
DO $$
DECLARE
    v_error_occurred BOOLEAN := false;
BEGIN
    BEGIN
        PERFORM app.request_temp_privilege('delete_contract', 0);
        v_error_occurred := false;
    EXCEPTION
        WHEN OTHERS THEN
            v_error_occurred := true;
            IF SQLERRM LIKE '%Длительность%' THEN
                RAISE NOTICE '✅ ТЕСТ 6.3 ПРОЙДЕН: Нулевая длительность
корректно отклонена';
                RAISE NOTICE '   Ошибка: %', SQLERRM;
            ELSE
                RAISE NOTICE '❌ ТЕСТ 6.3 ПРОВАЛЕН: Неожиданная ошибка: %',
SQLERRM;
            END IF;
    END;

    IF NOT v_error_occurred THEN
        RAISE NOTICE '❌ ТЕСТ 6.3 ПРОВАЛЕН: Нулевая длительность была
принята (не должно быть!)';
    END IF;
END $$;
```

```
--
============================================================
===
-- ИТОГОВАЯ СВОДКА
--
============================================================
===

DO $$
BEGIN
    RAISE NOTICE '';
    RAISE NOTICE '========================================';
    RAISE NOTICE 'РГЗ: Тестирование завершено';
    RAISE NOTICE '========================================';
    RAISE NOTICE '';
    RAISE NOTICE 'Проверьте результаты тестов выше.';
    RAISE NOTICE 'Все тесты должны быть помечены как ✅ ПРОЙДЕН';
    RAISE NOTICE '';
    RAISE NOTICE 'Для просмотра журнала временных привилегий:';
    RAISE NOTICE '  SELECT * FROM audit.temp_access_log ORDER BY
request_time DESC;';
    RAISE NOTICE '';
    RAISE NOTICE 'Для просмотра журнала вызовов функций:';
    RAISE NOTICE '  SELECT * FROM audit.function_calls WHERE
function_name = "request_temp_privilege" ORDER BY call_time DESC;';
END $$;

RESET ROLE;
```